

Mathematical Analysis and Development of an Algorithm for the Pre-Marshalling Problem

Jiaxu Yang
Gabriel Nørrelykke K. Kihara
Ilie Baciuc
Camilo Vergara
Sabin Ghimire
Cabdi Raxman Aadan

December 17, 2024

Abstract

The container relocation problem, or the premarshaling problem, is a well-known challenge in logistics and international trade, classified as NP -hard. This complexity requires heuristic or approximate algorithms to address the issue effectively. In this study, we developed an algorithm based on the Breadth-First Search (BFS) technique to minimize container relocations during the retrieval phase. BFS was chosen for its systematic exploration of nodes level by level, ensuring that optimal solutions are always provided. Additionally, we conducted a mathematical analysis of various potential container configurations to ensure a more in-depth understanding of the problem and an optimal path towards the solution. This work highlights our novel approach to improving container handling efficiency and reducing the negative consequences that affect the container shipping industry.

Contents

1	Theoretical Framework	7
1.1	Problem Definition	7
1.1.1	Mathematical Formulation	8
1.2	Computational Complexity	10
1.2.1	Search Problems	10
1.2.2	Decision Problems	10
1.2.3	Complexity Classes	10
1.2.3.1	P Class	13
1.2.3.2	NP Class	13
1.2.3.3	NP-complete Class	13
1.2.3.4	NP-hard Class	14
1.2.4	Introduction to the Block Relocation Problem	15
1.3	Algorithms for the Block Relocation Problem	15
1.3.1	Exact Algorithms, Approximate Algorithms, and Heuristics	15
1.3.2	Search Algorithms: BFS vs DFS	15
1.4	Literature Review	16
1.4.1	Review of Specific Papers	16
2	Mathematical Analysis	17
2.1	Initial Configurations	17
2.2	Target Configurations	18
2.3	Application of Mathematical Analysis	18
3	Resolution of the Algorithm	21
3.1	Problems, Limitations, and Challenges	21
3.2	A Perfect Algorithm	21
3.3	Code and Logic Description	22
3.3.1	Data Structures	22

3.3.2	Breadth-First Search (BFS)	23
3.3.3	Detection of Sorted Configuration	24
3.3.4	Move Generation	25
3.3.5	Top Container Identification	26
3.3.6	Full Column Verification	26
3.3.7	Container Movement	26
3.3.8	Optimal Path Reconstruction	27
3.3.9	JavaScript Code's Performance	28
3.4	Discussion and Improvements	32
4	Conclusion	32
5	Glossary	35
A	Initial Configurations Formula Explanation and Proof	37
A.1	Proof	41
A.1.1	Definitions and Notation	41
A.1.2	Fundamental Principles	42
A.1.3	Logic	46
B	Code	49
B.1	BFS Algorimth in JavaScript	49
B.2	BFS Algorimth in C	53
B.3	Additional data for code performance	62

The GAI declaration

1. Have you used GAI to receive feedback and criticism on text?
 - Yes, we used ChatGPT-4o to receive feedback in the following sections: The Abstract; the last paper of the literature review (Section 1.4), code data structures (Section 3.3.1), and BFS VS DFS (Section 1.3.2).
2. Have you used GAI to improve the language of the text?
 - Yes, we used Grammarly and ChatGPT-4o to check grammar mistakes.
3. Have you used GAI to generate code?
 - Yes, we used ChatGPT-4o to build and optimize most of the code sections, specifically large functions, and to identify and fix bugs in C and JavaScript codes. We also used ChatGPT-4o to help create the LaTeX code for figure 1, 2, 3, 4, 5, 6 and 7, and generate the skeleton code for all the tables in LaTeX.
4. Have you used GAI to write texts in the report (remember to cite the source in the relevant sections of the project report)?
 - Yes, we used ChatGPT-4o to generate the caption for figure 4 and the text for the last paper in the literature review. However, the information in the text was originally read and understood from the sources.
5. Have you used GAI to search for information?
 - No
6. Have you used GAI to get ideas and new perspectives?
 - No
7. Have you used GAI to suggest a research question?
 - No
8. Have you used GAI to generate figures?
 - No
9. Have you used GAI to figure out which sections the report should contain and what should be in each section?
 - No
10. Have you used GAI to make an abstract for the report (remember to cite the source in the relevant sections of the project report)?
 - No
11. Have you used GAI to find out how your empirical data should be analysed?
 - No
12. Have you used GAI to carry out analyses of your or other people's empirical data or data?
 - No

Introduction

Nowadays, international trade is a necessary part of the world. The transport of goods is carried out mainly using standardized containers, to allow efficient transportation across all types of transit. As the use of these containers increases, it is important to ensure that their transportation is as efficient as possible, as any delay or mistake can lead to economic losses [27]. Therefore, it is essential to minimize any oversight and avoid unnecessary use of resources and additional costs.

Delays can be caused by multiple factors, from weather conditions to operational inefficiencies. However, this report focuses on a critical factor: the arrangement of containers. Containers are typically stored in stacks at container yards, which are manipulated by Rail Mounted Gantry Cranes (RMGC) that can only move within a single layer, simplifying the problem. Containers are classified by priority and are usually linked to the order of destinations. Due to the lack of prior information, containers often arrive disorganized, causing higher-priority containers to be placed beneath lower-priority ones, complicating retrieval.

The mismanagement of containers in ports results in significant delays in the delivery of goods. This mismanagement increases operational costs and negatively affects the supply chain and consumers [27]. In addition, opportunities for slow steaming, a strategy in which vessels operate at lower speeds to reduce fuel consumption, are reduced, leading to increased fuel consumption and environmental pollution [22]. To address these challenges, it is crucial to develop solutions that improve container handling efficiency while supporting sustainable shipping practices.

Our goal is to develop an algorithm that efficiently organizes containers and minimizes the number of relocations while maintaining the correct retrieval order, thereby reducing the economic losses associated with delays. This will be further discussed in Section 1.

To develop this algorithm, we will focus on analyzing existing algorithms and techniques, conducting mathematical evaluations, and addressing the practical constraints of real-world port operations. Understanding these challenges is crucial to ensuring the algorithm’s feasibility and effectiveness in practice.

The theoretical foundation of this work lies in the study of search problems, which define a set of valid solutions for each case. In the context of container arrangement, finding the optimal sequence of moves can be viewed as a search problem where the objective is to find the most efficient solution within polynomial time. Efficient algorithms are those that can solve problems within a polynomial time, distinguishing them from inefficient algorithms that require exponential time [6]. By focusing on polynomially bounded search problems, we ensure that our approach remains practical and scalable for real-world applications.

This optimization is directly related to the broader theme of logistics efficiency, which has significant social impacts. Efficient container handling not only reduces delays and economic losses but also decreases fuel consumption of waiting vessels and lowers overall carbon emissions, contributing to more sustainable and economically viable shipping operations. According to recent statistics, inefficiencies in container handling can result in delays costing the global shipping industry millions annually, emphasizing the urgent need for improved logistic solutions [2].

Efficient container sorting in ports is particularly significant in the context of slow steaming. While slow steaming increases transit time, the time saved in ports due to optimized container handling can offset these delays. By minimizing unnecessary relocations, ports can reduce vessel

docking time and better align operations with slow steaming schedules. This alignment not only enhances economic efficiency but also promotes environmental sustainability. By addressing the Pre-Marshalling Problem through an optimized algorithm, this report aims to contribute to more efficient port operations, supporting the sustainability and profitability of international trade.

RQ: How can we develop an efficient algorithm for the Pre-Marshalling problem that would decrease economic losses?

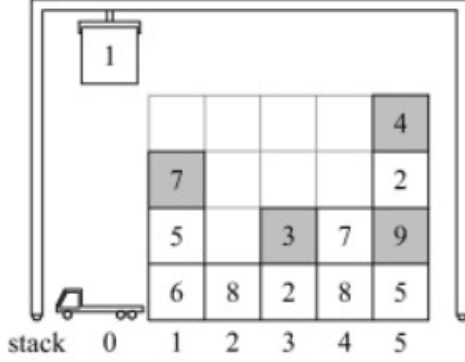


Figure 1: A RMGC sorting containers in a bay where the black containers are incorrectly placed containers. Image is from [12]

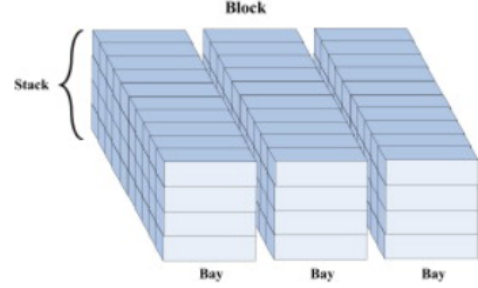


Figure 2: Image showing a port where the stacks form a bay and the bays form the block. Image is from [13].

1 Theoretical Framework

1.1 Problem Definition

Our goal is to solve the Block Relocation Problem (BRP), also known as the relocation or reordering problem of blocks. This problem requires us to find the best tactics and operational decisions to manage containers in ports [15]. In this section we will define the problem in a real world setting, by establishing some assumptions, definitions and rules towards the problem, that will simulate the scenarios close to reality.

Our assumptions and rules are similar to [14], here we are given a space in a port called a bay that has a finite number of columns referred to as n and a finite number of rows referred to as m for a given instance defined as a sequence of movements from the initial configuration to the final configuration (see figure 3). Each of these containers has the same size, dimension and occupies a certain percentage of the bay. The number of containers that occupy a certain number of rows in a column is called a stack, although both columns and stacks will mean the same thing in this paper. With each container having a priority number from 1 to z where z is the highest priority in the bay. These numbers indicate the container departure, with the smallest priority number departing first and the largest priority number being the last to depart, respectively. Our assumptions for the problem and the rules to simplify the problem are as follows:

1. For any crane to relocate a container, we need to have a height limit equal to number of rows n , and a width limit equal to the number of columns m (see figure 1).

2. We assume the crane can only choose and collect one container at a time meaning when we develop the algorithm only one container can be picked at a time.

Unfortunately, computers do not have a visual concept of things and a sense of perspective. Therefore, even the most obvious of instructions to a person, is not obvious to a computer. Hence, such basic rules as, for example, gravity must be introduced as an instruction in the code for the proper functionality.

Our strategy to solve the BRP is to implement an exact algorithm (see section 1.3.1) that always gives an optimal solution. Unfortunately, our exact algorithm will be slow while finding the optimal solution for larger configurations. So we have limited the problem to find the optimal solution without consideration of the time taken to find it.

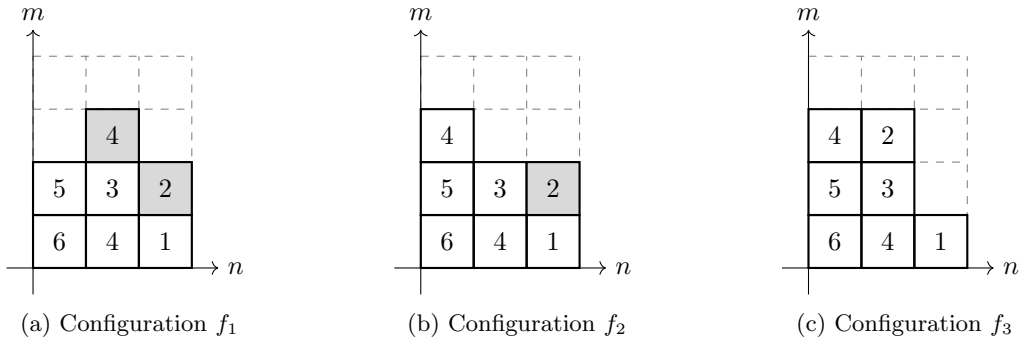


Figure 3: The image from [14], shows the minimal relocation's to sort the configuration. The misplaced containers are labeled in gray, the sorted in white, where m is the rows and n are the columns. The height limit is 4 and a width limit is 3. Here we demonstrate our goal of finding minimum number of moves to a solution, while following the rules and assumptions we made.

1.1.1 Mathematical Formulation

There are multiple ways to define the problem mathematically, but we will adopt a straightforward approach (Some symbols and definitions are in section 5).

We define a container as an abstract entity occupying a discrete position within a storage structure. Formally, each container $C_i \in \mathcal{C}$ is characterized by:

$$C_i = (i, p_i)$$

Where:

1. Unique Identity:

$$i \in \{1, 2, 3, \dots, c\}$$

is a unique identifier that distinguishes C_i from other containers in the set $\mathcal{C}$.

2. Discrete Position:

$$(x_i, y_i) \in \mathcal{P}$$

where $\mathcal{P}$ is the set of positions in the bay, defined as:

$$\mathcal{P} = \{ (x, y) : x \leq n, y \leq m, x, y \in \mathbb{N} \}$$

with n columns and m rows.

3. Priority:

$$p_i \in \{1, 2, 3, \dots, z\}$$

representing the priority level of container C_i .

Containers sharing the same priority are indistinguishable. The number of containers for each priority i ($1 \leq i \leq z$) is j_i , such that:

$$\sum_{i=1}^z j_i = c$$

A valid configuration is an assignment $f : \mathcal{C} \rightarrow \mathcal{P}$ satisfying:

1. Injectivity: No two containers occupy the same position:

$$\forall C_1, C_2 \in \mathcal{C}, C_1 \neq C_2 \implies f(C_1) \neq f(C_2)$$

2. Gravity Constraint: If a container is at position (x, y) with $y > 1$, there must be a container directly below it at position $(x, y - 1)$:

$$\forall C \in \mathcal{C}, y_i > 1 \implies \exists C' \in \mathcal{C} \text{ such that } f(C') = (x_i, y_i - 1)$$

3. Completeness: All containers are placed within the bay; that is, f is a total function.

Our objective is to transform an initial valid configuration f_{initial} into a final configuration f_{final} that satisfies the following Priority Constraint:

In the final configuration, containers in each column are stacked in non-decreasing order of priority from bottom to top. Formally, for each column x and for all y such that $1 \leq y < s_x$, where $s_x = m - e_x$ is the number of occupied positions in column x , and e_x is the number of empty positions in column x , we have:

$$p(f^{-1}(x, y)) \leq p(f^{-1}(x, y - 1))$$

Since f is a function that associates a container to a position, f^{-1} is a function that associates a position to a container, and the function p returns the priority of the container, so the highest priority should always be at the top and the lowest priority (the largest number) should be at the bottom.

Constraints:

1. Movement Constraint: At each step, only the top container of any column can be moved, and it can only be placed on top of another column without exceeding the maximum height m .

2. Gravity Constraint Maintenance: After each movement, the gravity constraint must remain satisfied.
3. Stack Height Constraint: The height of any column cannot exceed m at any time:

$$\forall x \in \{1, 2, 3, \dots, n\}, s_x \leq m$$

Objective Function:

Minimize the total number of container movements required to reach f_{final} from f_{initial} . Formally, we seek a sequence of configurations $\{f_t\}_{t=0}^T$, where $f_0 = f_{\text{initial}}$, $f_T \in f_{\text{final}}$ (It should be remembered that f_{final} is a family of solutions since there is usually more than 1 sorted configuration), and each $f_t \in \mathcal{G}_{\text{valid}}$, such that:

$$\min T$$

subject to the constraints above.

1.2 Computational Complexity

1.2.1 Search Problems

A search problem is a specification of a set of valid solutions (including empty ones) for each possible instance. Thus, for any given instance, the task is to find a solution to the problem or determine that no solution exists. Search problems correspond to the idea of “solving a problem”. For example, find a valid solution of a given system of equations or sorting a list of numbers [6].

1.2.2 Decision Problems

A decision problem must determine if the instance belongs to the specified subset. For example, to determine if a given natural number is a prime number [6].

Being able to determine whether or not a solution exists is a prerequisite to being able to solve the corresponding search problem. In general, decision problems refer to the natural task of making binary decision, a task that is common in daily life (e.g, determining whether a traffic light is red) [6].

1.2.3 Complexity Classes

In computer science, problems are organized into complexity classes based on the amount of time and memory an algorithm requires to solve the problem and verify the solution. Polynomial functions grow at a manageable rate, meaning their values increase predictably as input sizes becomes larger [1] (see figure 4 and 5).

Efficient algorithms are generally defined as those that can solve a problem within a number of steps that is a polynomial function of the input size, reflecting moderate growth. In contrast,

slower algorithms may need exponential time, which grows much faster and quickly becomes impractical for large inputs. Polynomial functions, therefore, are used to set the standard for efficient time bounds, representing a robust, moderately growing measure across various models of computation [1].

It is helpful to distinguish between two types of intractability under this definition. The first, and most commonly considered, is when a problem is so complex that finding a solution requires exponential time. The second type occurs when the solution itself is so large that it cannot be expressed in a form whose length is bounded by a polynomial function of the input size [1].

The second type of intractability can be seen in a version of the traveling salesman problem (find the shortest route for a salesman to visit a number of cities) that includes an additional parameter, B , and asks for all possible tours with a total length of B or less. In this case, it's possible to create problem instances where there are exponentially many tours within the given length limit, making it impossible for any polynomial-time algorithm to list them all [1].

There is general consensus that a problem is not truly 'well-solved' until a polynomial-time algorithm has been found for it. Therefore, a problem is described as intractable (in worst case) if it is so difficult that no polynomial-time algorithm can possibly solve it [1].

Complexity classes are determined by the "external behavior" of algorithms—how they perform in terms of time and memory—rather than by the internal characteristics of the problems they solve. This focus on performance allows complexity classes to be broadly applicable across different types of problems and computation models [1].

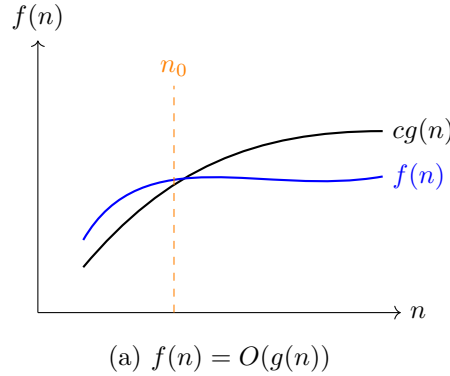


Figure 4: Illustration of O -Notations. This graph illustrates how $f(n)$, represented by the blue curve, and $cg(n)$, represented by the black line, relate in terms of complexity growth. As shown, $f(n) \leq cg(n)$ for sufficiently large n , demonstrating the concept of asymptotic upper bounds in O -notation [3].

Comparison of Linear, Polynomial and Exponential Growth

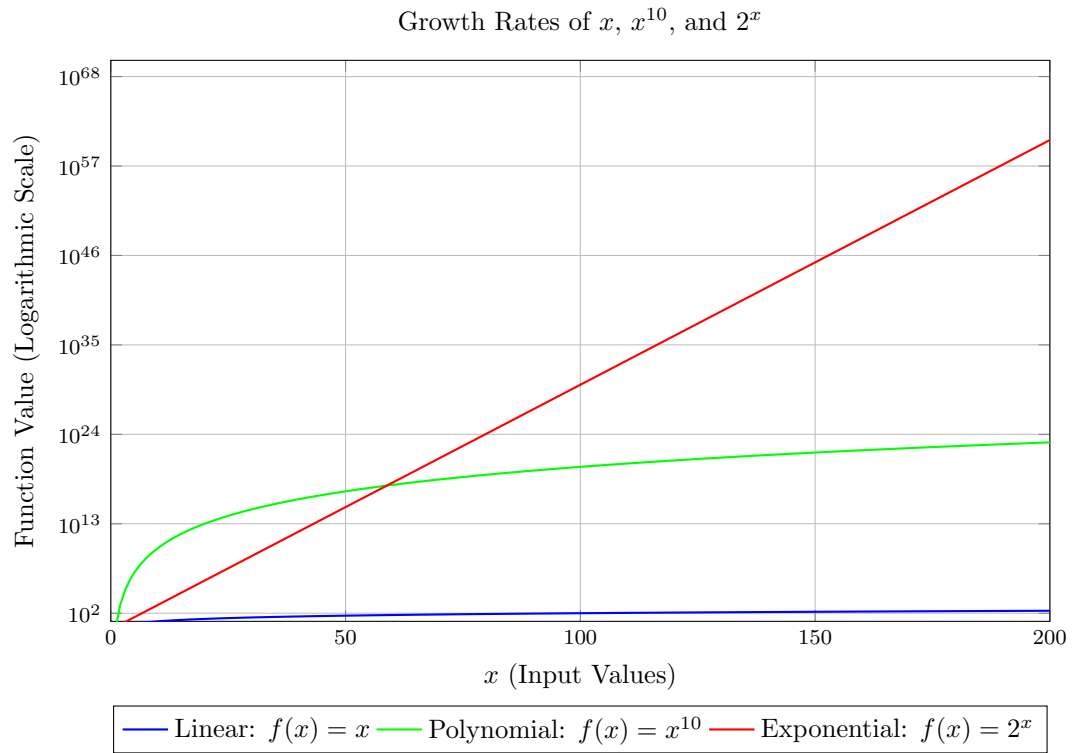


Figure 5: Comparison of growth rates for three functions: a linear function ($f(x) = x$), a polynomial function ($f(x) = x^{10}$), and an exponential function ($f(x) = 2^x$). The linear function grows at a constant rate, the polynomial function grows faster, and the exponential function dominates at larger values of x .

1.2.3.1 P Class

The class of search problems known as *PF* (Polynomial-time Find) consists of problems that can be solved efficiently, meaning the solutions can be found within a reasonable timeframe. These problems are considered tractable because they are both mathematically solvable and computationally feasible in real-world scenarios. Additionally, the size of the solutions is bounded by a polynomial function of the input size, and the solutions can be verified in polynomial time. [6].

Examples of these problems include finding solutions to systems of linear equations, sorting lists of numbers, locating patterns within strings, and finding the shortest path in graphs, among others. These problems can all be addressed in polynomial time, demonstrating their practical tractability [6].

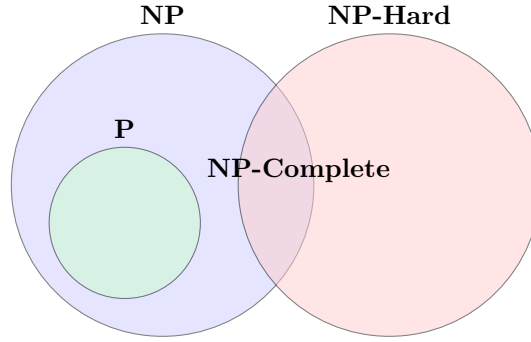
1.2.3.2 NP Class

NP stands for Non-Deterministic Polynomial time. Is another class of search problem. Problems in this class are hard to solve, although the solutions are easy to check whether they are correct. In other words, even though finding the solution might take a lot of time, checking if a given solution is correct can be done quickly. It also highlights the distinction between ‘solving’ and ‘checking,’ showing that finding a solution can be much harder than verifying it [6].

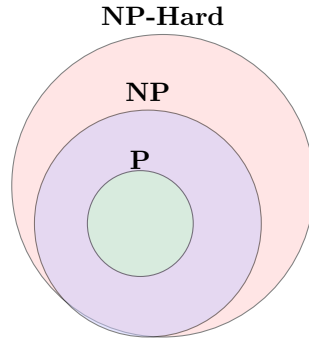
Examples of *NP* problems include finding the prime factorization of a given composite, finding a truth assignment that satisfies a given Boolean formula or the *Traveling Salesman Problem*, an optimization problem where the objective is to find the shortest possible route that visits each city exactly once and returns to the starting point [6, 17].

1.2.3.3 NP-complete Class

NP-complete (abbreviated *NP-C* or *NPc*) is theory applied to only decision problems. Not all the problems whose solutions can be verified in polynomial time (*P*) are solvable in polynomial time (*NP*) as well ($P \neq NP$). Proving that the solution can be verified in polynomial time is easy, although proving that the problem cannot be solvable in polynomial time is very difficult, because it would be required to demonstrate the nonexistence of efficient algorithms to solve the problem. Therefore, the theory of *NPc* provides a weaker, but practical approach to $P \neq NP$. *NPc* are the hardest problem in *NP*, so if any *NPc* problem can be solve in polynomial time, then all the problems in *NP* can be solve in polynomial time ($P = NP$). This way proving that a problem is *NPc* would strongly indicate that a problem is likely intractable (not solvable in polynomial time) [1].



(a) NP, P, NP-Hard, where $NP - \text{Hard} \cap NP = NP - \text{Complete}$. In this image it demonstrates the hypothesis of $NP \neq P$ is unknown whether it is true or false. Although most researcher believe that this is true [1]



(b) Subset relationship: NP-Hard, NP, and P. This image shows the hypothesis of $NP = P$, although it is unknown whether is true or false. Unlike $NP \neq P$ most scientists do not believe this is the case [17].

Figure 6: Venn diagrams depicting computational complexity sets and theories. Both figures are from [20]

1.2.3.4 NP-hard Class

NP -hard stands for non-deterministic polynomial-time hard and refers to a class of problems that are at least as hard as the hardest problems in NP . A problem H is NP -hard if a NP -complete problem L can be reduced to H in polynomial time. This means that if there is a polynomial time algorithm that can solve a NP -hard problem, then all the NP problems can be solve in polynomial time, making $P = NP$ [17].

NP -hard problems are not limited to decision problems; they can also include search and optimization problems (see Figure 6a). An example of a NP -hard problem that is also NP -complete is the *Subset Sum Problem*, where, given a set of integers, the goal is to determine whether any non-empty subset sums to a specific target value (including zero). Another example of an NP -hard problem is the *Traveling Salesman Problem*.

1.2.4 Introduction to the Block Relocation Problem

From the discussion in this section, we can state that the Pre-Marshalling Problem (PMP), also known as the Block Relocation Problem (BRP), is *NP*-hard [8]. Caserta et al. [8] demonstrated that if the BRP could be solved efficiently, then the Mutual Exclusion Scheduling (MES) problem—previously shown to be *NP*-complete, could also be solved, confirming that the BRP is at least *NP*-hard.

It is important to note that the BRP has multiple variants and can be approached in different ways. Several highly efficient algorithms already exist, employing techniques such as A* and IDA*, as shown in [14]. As we will demonstrate in the literature review (Section 1.4), this problem has been extensively studied. Therefore, our approach will focus on an analytical perspective rather than a purely technical one. While various mathematical formulations, such as the one proposed in [18], are available, our work will emphasize detailed mathematical analysis and the development of an optimal algorithm.

1.3 Algorithms for the Block Relocation Problem

1.3.1 Exact Algorithms, Approximate Algorithms, and Heuristics

To understand the literature review, we first need to explain the differences between exact algorithms, approximate algorithms, and heuristics. Exact algorithms always find the optimal (best) solution, but not always in reasonable time as they will keep searching through the possibilities until an optimal solution is found. Therefore, they are terrible at solving *NP*-complete and *NP*-hard problems, as the time taken to find the solution is too high. They can be optimized usually through narrowing the search by adding rules and conditions to reduce the running time[3, 4]. To solve the issue where exact algorithms take too long to find an optimal solution in *NP* problems, approximation algorithms are used. Unlike exact algorithms, they try to find a solution that is closest to the optimal solution in reasonable time. On the other hand, heuristics are a set of instructions derived from data or experiences that shorten the route to a solution. They can exist as algorithms under the same category as approximate algorithm, but they commonly have a lower optimality than approximate algorithms, measured by approximation ratio (the deviation from the approximate solution and optimal solution)[3].

1.3.2 Search Algorithms: BFS vs DFS

The Breadth-First Search (BFS) algorithm traverses a graph in a forward breadth-like motion, exploring all nodes at a reachable distance k . The unexplored nodes are placed in a queue (see Section 3.3.1), and once a node is explored it is removed from the queue. This is done to prevent the BFS from revisiting explored nodes in k . Once all nodes are explored in k , it moves to the next set of nodes closest to k . This is repeated until a solution is found. Depth-First Search (DFS) algorithm also traverses a graph, although it explores nodes in one specific branch, and when all the nodes in the branch are explored it returns to the previous node and moves to the adjacent unexplored node and continues the search. This is repeated until a solution is found. The DFS can visit a node only twice, one for exploration and the other for analysis. Both DFS and BFS indicate whether the nodes have been explored, and determine whether the unexplored nodes are reachable. They both have the same running time as $O(|V| + |E|)$ where $|E|$ is the number of connections between nodes and $|V|$ is the number of nodes [3].

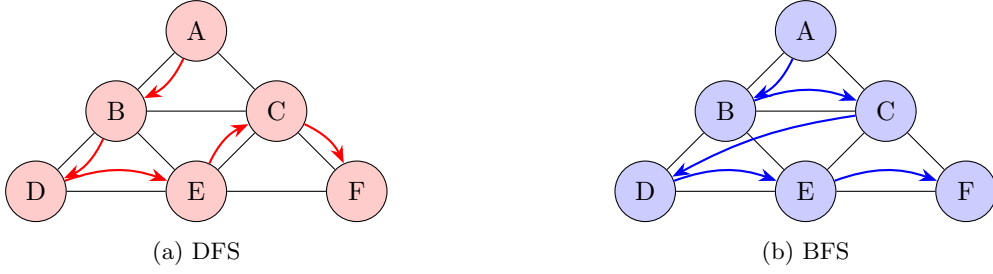


Figure 7: Comparison between DFS and BFS in the same graph inspired by [3].

The DFS diagram in red shows how it traverses through the full depth of one branch and then backtracks and traverses again through a new branch. The BFS diagram in blue shows how it traverses through the levels with each node at a given level being searched one by one. Both will keep on traversing in their respective manner until a solution is found.

1.4 Literature Review

In this section, we will review various algorithms and heuristics that have been developed to solve the Block Relocation Problem (BRP), focusing on both approximate and exact methods. The BRP is a well-studied problem, with various approaches, and the relevant papers for our report are [14], [15], and [18].

1.4.1 Review of Specific Papers

The first and only approximate algorithm we will review is from Tierney et al. [14], utilizing the A* and IDA* with the implementation of branching and relocation rules to optimize the speed. The branching rules include a maximum depth search, a lower bound (maximum number of moves), and tie breakers where the one that reduces the most amount of relocation and has lowest priority container at the top of the stack is chosen. Relocation rules disallow transitive moves (adding moves to the solution), repeating or redundant moves, and limiting the occupation of empty stacks. These algorithms were benchmarked with state-of-the-art A*, and tested with different branching and relocation rules to evaluate their behavior using manipulated CV instances, EMM, and BF datasets from [5], [10], and [7] respectively. These tests provided information on how the algorithm operated with and without specific rules and in various scenarios. The results for the IDA* and A* varied in specific instances and rules, making their performances unreliable when solving general instances compared to specific cases. However, the algorithm demonstrate incredible improvement in speed and solution optimality.

The first exact algorithm we will review is presented in paper [15] by Tanaka and Takii who developed a tree-based algorithm, used to solve the unrestricted BRP, this is a version of BRP that allows relocating containers from any position [21]. They utilized a branch-and-bound algorithm with sets of theorems and dominance (branching) rules to remove redundant moves. These rules are applied to the paired relocations where 2 containers displace one another, single relocations, where 1 container displaces another container, and relocations that have no effect on other relocations and freeing of the target containers. If no relocations can occur due to the rules, then illegal moves are executed, and the layout is re-analyzed. Lower bound to the number of moves to reduce exploration of redundant nodes. The algorithm was benchmarked with other exact algorithms and tested using CVS instances from [9] and [5], and ZQLZ instances from

[11]. The algorithm does well in finding the optimal solution for all instances and the dominant rules improved its speed. Unfortunately, the time required increases exponentially for complex instances, with some instances remaining unsolved within the time limit.

The final paper we will review is by Tanaka and Mizuno [18], follow-up to [15]. Here, they address the Block Relocation Problem with a Stowage Plan (BRPSP), optimizing container movements within a yard to meet specific vessel loading orders while minimizing relocations. Key concepts include precedence cycles (the sequence of containers relocated to the same stack), which dictate container access order and highlight conflicts to reduce unnecessary moves. Three lower bounds (LB2c, LB2c4c, and LBr) estimate minimum relocations by accounting for varying levels of dependencies and space assumptions. An exact algorithm is proposed using branch-and-bound using the same dominance rules and theorems as [15] to enhance efficiency. Experiments were conducted using derived instances from [16]. The results showed that this approach is an improvement from [15].

The time limit in [15], reduces the accuracy of the running times growth as the input size grows, preventing proper evaluation of the algorithm efficiency. This time limit also prevents us from determining whether the lower bound allows the algorithm to always find an optimal solution for every scenario. The follow-up paper [18] addresses this issue by relaxing these lower bounds to solve the limitation. The branch-and-bound branching rules improved the runtime of the algorithm, offering a way of approximating the direction of a solution in an exact algorithm. The use of cyclic traversal rules in [18] allowing fewer relocations and node traversal.

To conclude, our approach will utilize [14] understanding of the problem, and [15], and [18] approach of applying exact algorithms to solve the BRP, with our algorithm performing a traversal through a graph. However, we will not perform any optimization to our algorithm (BFS) to ensure we have full optimality regardless of runtime.

2 Mathematical Analysis

In this section we will delve into the mathematical part of the problem, since it is a fundamental part to create an efficient and functional algorithm, besides the fact that this problem is directly related to different branches of mathematics such as graph theory, computational complexity theory, combinatorial optimisation and others. Although we will not analyse the whole problem in all these mathematical disciplines, it should be fundamental to understand at least 2 key points of the problem that are extremely useful, not only for the current research but also for future research, which are to understand the number of initial configurations and the number of target configurations. Knowing in detail how they behave will allow us for example, to generate a graph space where each node represents a specific configuration and these nodes are connected to other nodes where the configuration is only 1 move away, in this situation knowing well how many nodes there are in total and how many of these are the target can greatly simplify the solution of the problem.

2.1 Initial Configurations

Knowing the number of initial configurations is fundamentally important, as this number allows us to understand the size of the space we are working within and has multiple applications, which we will explain later. The formula to determine the exact number of initial configurations is as follows:

$$\zeta(m, n, c, \{j_i\}_{i=1}^z) = \frac{c!}{\prod_{i=1}^z j_i!} \sum_{k=0}^n (-1)^k \binom{n}{k} \binom{mn - c - k(m+1) + n - 1}{n-1} \quad (1)$$

The proof can be found in Section A.

For instance, if we want to calculate the number of configurations with 5 rows and 3 columns, using 6 containers as follows: $\zeta(5, 3, 6, \{1, 2, 3\})$. We substitute the variables into the formula:

$$\zeta(5, 3, 6, \{1, 2, 3\}) = \frac{6!}{1!2!3!} \left(\binom{3}{0} \binom{11}{2} - \binom{3}{1} \binom{5}{2} + \binom{3}{2} \binom{-1}{2} - \binom{3}{3} \binom{-7}{2} \right)$$

We simplify and eliminate the binomial terms with negative indices:

$$\zeta(5, 3, 6, \{1, 2, 3\}) = \frac{720}{12} \left(\binom{11}{2} - 3 \binom{5}{2} \right)$$

The binomials are converted into their factorial expressions:

$$\zeta(5, 3, 6, \{1, 2, 3\}) = \frac{720}{12} \left(\frac{11!}{2!9!} - 3 \frac{5!}{2!3!} \right)$$

We compute the value manually or using a calculator, obtaining:

$$\zeta(5, 3, 6, \{1, 2, 3\}) = 1500$$

It is clear that this is a relatively large number, and it increases as m , n , and z grow. Additionally, it also grows when the value of c is neither too large nor too small, though this will be discussed in more detail in the following sections.

2.2 Target Configurations

A target configuration is one in which each of its columns satisfies the condition that all containers have a container with higher priority above them or have no container. It is quite evident that the number of target configurations is equal to or less than the number of total configurations, since all target configurations can also be initial configurations, but not all initial configurations are target configurations. Therefore, we will denote the number of target configurations as α .

Unfortunately, We have not been able to develop a formula that allows determining the exact number of solutions due to the complexity of the problem and the factors that may be involved. Therefore, the development of the formula is left as an exercise to the reader.

2.3 Application of Mathematical Analysis

In this section, we will discuss the applications, utility, and how it relates to the Block Relocation Problem (BRP). We begin by imagining a way to interpret the problem: we can construct a

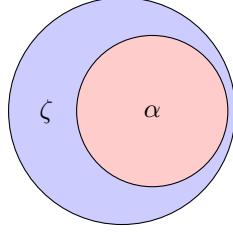


Figure 8: α is a subset of ζ

graph with all possible configurations given by formula (1). Each node represents a unique initial configuration and is connected to all neighboring configurations that have only one relocated container following the rules and restrictions.

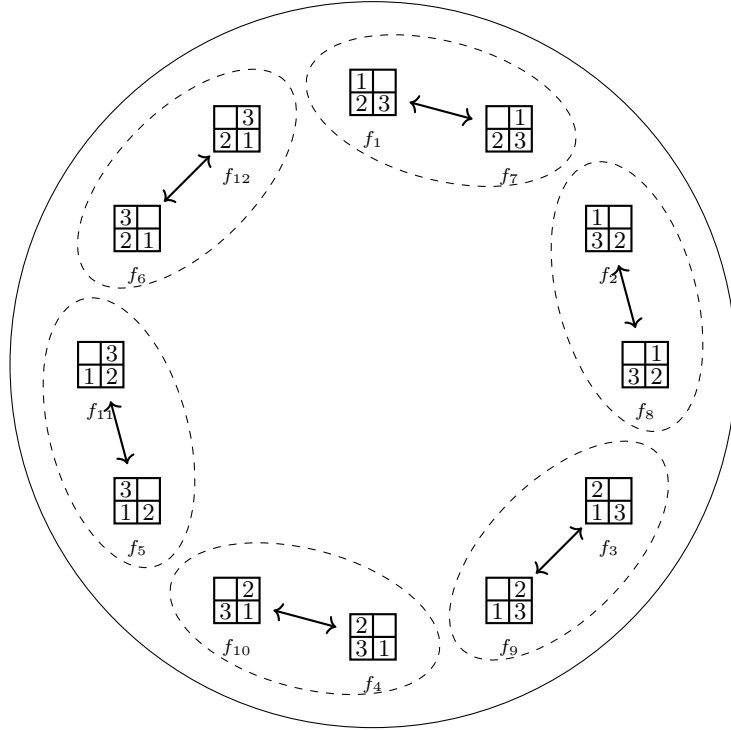


Figure 9: All possible initial configurations with 2 columns, 2 rows and 3 containers of different priorities expressed in formula (1) as $2R2C(1, 1, 1)$. The initial configurations that can transition into one another are grouped into "subsets".

From the diagram 9, we can extract essential information for constructing our algorithm. Specifically, any initial configuration belongs to a particular subset ($\mathcal{H}_i$) within the set of valid configurations ($\mathcal{G}_{valid}$). Due to the imposed restrictions, transitions between configurations are confined to within the same subset, meaning that no valid transformation allows movement to a different subset.

This example is deliberately limited by permitting only one empty space, which significantly restricts the possible movements of containers. The number of empty spaces is crucial in determining the flexibility of transitions between configurations:

- Limited Empty Spaces: With only one empty space, the number of possible movements is highly restricted, resulting in multiple distinct subsets within $\mathcal{G}_{valid}$.
- Increased Empty Spaces: As the number of empty spaces increases, the number of subsets decreases. Eventually, when the number of empty spaces is sufficient, the subset $\mathcal{H}_i$ encompasses the entire set $\mathcal{G}_{valid}$. At this point, any configuration can transition to any other configuration within the set, indicating maximal flexibility.

In most real-world scenarios, the number of empty spaces is not as restrictive as in our initial example. Consequently, the subset $\mathcal{H}_i$ tends to be equal to the entire set $\mathcal{G}_{valid}$, allowing for a broader range of transitions between configurations.

This observation leads to a crucial insight: while formula (1) does not always specify the exact number of configurations the algorithm must evaluate, it provides a meaningful upper bound. Specifically, with enough empty spaces, the subset $\mathcal{H}_i$ becomes identical to the entire set $\mathcal{G}_{valid}$, ensuring that all possible configurations are accessible.

Intuitively, for any given set of variables, there exists at least one configuration where the containers are ordered correctly. However, the existence of such a configuration does not guarantee that it resides within the same subset as the initial configuration. From this, we can derive several important conclusions:

1. If no target configuration exists within the same subset as the initial configuration, the algorithm will not find a solution.
2. There exists at least one subset within $\mathcal{G}_{valid}$ that contains a target configuration, implying that all configurations within this subset have a valid solution path.
3. When the number of empty spaces is sufficiently large, making the subset $\mathcal{H}_i$ equal to the entire set $\mathcal{G}_{valid}$, a solution must necessarily exist or in other words, any given configuration can be converted into another configuration.

Understanding the distribution of target configurations across subsets would be immensely valuable. It would allow us to determine the maximum number of subsets containing solutions and the number of configurations within each subset, providing crucial insights for optimizing the search algorithm and deepening our understanding of the problem. Although we have not yet developed these specific formulas, we have identified some interesting upper and lower bounds, which are more appropriately addressed in a separate study.

Clearly, all the statements in this section require formal, rigorous mathematical proofs. However, since these proofs are highly intuitive and not essential for understanding the utility and applications of the formula, we will not provide them here.

3 Resolution of the Algorithm

In this section, following the mathematical analysis, we will attempt to construct a resolution algorithm for the main problem. Unlike other studies, the focus here is not on providing a perfect solution or optimizing existing algorithms, but on taking a more constructive approach. The goal is to design an algorithm that follows straightforward and logical instructions, allowing us to reconstruct the step-by-step progress toward a solution.

Given that efficiency and optimization are not our primary concerns, we will focus on developing an algorithm using *JavaScript* code that can be interpreted (read) by *Node.js* environment.

3.1 Problems, Limitations, and Challenges

It is essential to address and consider all the limitations and challenges one might face when developing this algorithm. The first noteworthy aspect of this problem, which makes it more complex than others (e.g., the Tower of Hanoi), is that there is no predefined initial and final configuration where only a few variables are modified. In this case, the number of variables is much larger, and even if all the variables are known, there can be multiple initial and target configurations.

Given an initial configuration, it is not possible to determine which target configuration will be reached. Even if it is known, there are still multiple ways to achieve it.

Furthermore, any algorithm must ensure that it functions correctly. By "functions," we mean that the algorithm must find a solution if one exists. This might seem simple, but it is not, as some algorithms follow predetermined strategies that may favor certain target configurations. This can result in loops or, worse, in an incorrect conclusion that no solution exists instead of exploring alternative target configurations.

Finally, it is important to consider that there may be multiple solutions that require the same number of moves. Although this is not necessarily a limitation or an issue, in real-world scenarios, container movement is not the only factor that affects time and energy consumption; distance may also play a role. Therefore, even if a perfect algorithm provides a solution, it could be beneficial to introduce a secondary algorithm that minimizes the total movement distance. However, this secondary step should only be applied when solving the main problem - finding a valid configuration with minimal moves - is computationally efficient and does not require significant resources. Certainly, if one were to be meticulous, this factor would be initially taken into consideration, but for simplicity in addressing the main problem, we will assume it to be insignificant. Incorporating this consideration would considerably alter the problem analysis and could provide an interesting basis for future research.

3.2 A Perfect Algorithm

In problems of this nature, one might wonder whether perfect algorithms exist. The answer is, in fact, "yes." In this context, an algorithm is considered perfect if it provides a solution with the fewest possible moves and determines whether a solution exists. Generally, it must be both optimal and precise. Currently, several algorithms satisfy both requirements, some of which are mentioned in Section 1.3.2.

Creating a precise algorithm is relatively straightforward. For instance, an algorithm that randomly relocates containers could eventually find a solution if one exists. However, this approach is highly inefficient and has a critical flaw: in the absence of a solution, it would search indefinitely.

In this study, we will use a well-known algorithm called *BFS* (Breadth-First Search). The primary reason for choosing BFS over algorithms like DFS or A* is that BFS guarantees that the first solution it finds is also the shortest possible. By systematically exploring configurations level by level, BFS ensures that all configurations requiring the same number of moves are examined before progressing to those requiring more moves. Consequently, if a solution is found in, for example, five moves, we can state that no solution exists with fewer than five moves. Of course, readers familiar with graph search algorithms are aware that BFS requires more computational resources as the complexity of configurations increases (i.e., as graphs grow overwhelmingly large). This is entirely true. However, BFS is straightforward to understand and well-suited for analyzing the problem without the additional complexities introduced by heuristic-based or more efficient algorithms, such as A*.

3.3 Code and Logic Description

3.3.1 Data Structures

Before introducing the code logic, it is important to explain the data structures used, as data structures and algorithms (DSA) go hand in hand. For readers unfamiliar with this topic, a data structure is a method of organizing and storing data, which is essential for creating fast and efficient algorithms [3]. In our code, these data structures have specific purposes, represented as follows:

1. **Arrays**

An array is a data structure used to store multiple elements. In our code, arrays are used to represent the grid configuration compactly, making it easy to access and modify the state of the containers. Arrays also help store the initial configuration, intermediate configurations, updated states after each move, and allow precise indexing to locate containers and their destinations within the grid. (e.g., lines 4–9) [3]

2. **Queues**

A queue is a data structure that holds multiple elements, following the "first in, first out" (FIFO) policy: the first element added is the first to be removed. The BFS algorithm uses queues to manage unvisited nodes, removing each node once it has been visited at a given distance [3].

3. **Maps**

A map is a data structure that stores and retrieves data based on unique key-value pairs. A key is a single unique identifier (e.g., "age"), and the specific data associated with the key is called a value (e.g., "25"). In our code, the keys represent the visited nodes (configurations) and the values are the parent nodes (configurations). These parent nodes help reconstruct the path from the initial configuration to the solution. The parentMap in the code tracks the relationship between configurations during the breadth-first search (BFS) traversal [23].

4. **Sets**

A set is a data structure that stores collections of unique and distinguishable elements known as members. A set automatically ensures that there are no duplicate elements. If an element is already a member of the set, adding it again has no effect. In the code, the visited nodes are stored in a set to ensure uniqueness and avoid duplication [3].

It must be emphasized that once a solution is found, all connections are removed except for those that form the shortest path to the solution. This results in a breadth-first search (BFS) tree - a hierarchical data structure where nodes represent configurations and edges represent moves between them. Unlike a general graph, a tree has no cycles, and there is exactly one path between any two nodes [24].

We have developed two versions of the algorithm: one in JavaScript and the other in C. The choice of JavaScript is motivated by its support for external libraries, relatively fast compilation speed, and simpler syntax. On the other hand, the choice of C is driven by its superior execution speed compared to JavaScript (C is indeed one of the fastest programming languages in the world), which further improves the algorithm's efficiency. Nevertheless, to avoid tiring the reader, the following sections will focus only on the JavaScript implementation. However, both versions can be found in the Appendices (Sections B.1 and B.2).

3.3.2 Breadth-First Search (BFS)

The BFS algorithm operates as follows:

1. Initialization:

- The algorithm begins by checking if the initial configuration is already sorted.
- If the configuration is sorted, the algorithm terminates immediately.
- Otherwise, the initial configuration is added to the state queue and the visited set. Since it has no predecessor, it is associated with `null` in the parent map.

2. Main Loop:

- Dequeue the front configuration for analysis (line 2).
- Check if the dequeued configuration is sorted (lines 4–6):
 - If it is, reconstruct the path using the parent map and terminate the algorithm.
- Generate all valid moves (new configurations) from the current configuration (line 9).
- For each new configuration (lines 11–18):
 - Check whether it has already been visited (line 13).
 - If it has not been visited:
 - Add it to the visited set (line 14).
 - Record the current configuration as its predecessor in the parent map (line 15).
 - Add it to the state queue (line 16).

The main loop is implemented as:

```

1 while (queue.length > 0) {
2     let currentConfig = queue.shift();
3
4     if (isSorted(currentConfig)) {
5         reconstructPath(currentConfig);
6         return;
7     }
8
9     let possibleMoves = generateMoves(currentConfig);
10
11     for (let move of possibleMoves) {
12         const moveStr = JSON.stringify(move);
13         if (!visited.has(moveStr)) {
14             visited.add(moveStr);
15             parentMap.set(moveStr, JSON.stringify(currentConfig));
16             queue.push(move);
17         }
18     }
19 }

```

3.3.3 Detection of Sorted Configuration

To verify whether a configuration is sorted, the algorithm checks that each *column* contains containers in **non-decreasing order** (i.e., each container is greater than or equal to the previous one) from top to bottom. Configurations are represented in a **1D array**, where the grid is "flattened" column by column.

In this representation:

- i is the column index (ranging from 0 to $n - 1$, where n is the number of columns).
- j is the row index (ranging from 0 to $m - 1$, where m is the number of rows).
- m is the number of rows in the grid, which determines the offset between elements in the same column.

The value at position $i \times m + j$ in the array corresponds to the container at **row** j and **column** i of the grid.

The verification process involves two **for** loops:

- The **outer loop** iterates over the columns (i).
- The **inner loop** iterates over the rows (j) of each column.

For each column, the algorithm ensures that every container value is greater than or equal to the one above it. If any column fails this check, the configuration is considered unsorted.

The function to check if a configuration is sorted is:

```

1 function isSorted(config) {
2     for (let col = 0; col < n; col++) {
3         let prev = 0;
4         for (let row = 0; row < m; row++) {
5             let index = col * m + row;
6             if (config[index] >= prev) {
7                 prev = config[index];
8             } else {
9                 return false;
10            }
11        }
12    }
13    return true;
14 }

```

3.3.4 Move Generation

Generating possible moves is crucial for correctly constructing the search tree. The process is as follows:

- Identify the top container in each column that can be moved.
- For each possible target column:
 1. Check that the target column is not full and is different from the source column.
 2. Check that the move follows the problem's rules (e.g., priority order of stacking rules).
 3. Simulate the move and create a new configuration.
 4. If the new configuration has not been visited, add it to the queue and update the corresponding data structures.
- Ensure that redundant or cyclic configurations are avoided to prevent unnecessary computations.

This process ensures that all valid moves are considered while following the rules of the problem, helping the BFS algorithm explore the search space properly and efficiently.

The move generation function is implemented as:

```

1 function generateMoves(config) {
2     let moves = [];
3     for (let fromCol = 0; fromCol < n; fromCol++) {
4         const topContainer = getTopContainer(config, fromCol);
5         if (topContainer !== null) {
6             for (let toCol = 0; toCol < n; toCol++) {
7                 if (toCol !== fromCol && !isColumnFull(config, toCol)) {
8                     let newConfig = moveContainer(config, fromCol, toCol)
9                     ;
10                    moves.push(newConfig);
11                }
12            }
13        }
14    }
15 }

```

```

14     return moves;
15 }

```

3.3.5 Top Container Identification

This function identifies the first accessible container in a column. It scans the column from top to bottom and returns the first non-zero value it encounters, along with the index of that value.

If the column contains only zeros (i.e., no accessible containers), the function returns null. In this case, the algorithm skips this column and moves to the next one during the move generation process. This ensures that only valid source columns with accessible containers are considered, avoiding unnecessary computations for empty columns.

```

1 function getTopContainer(config, col) {
2     for (let row = 0; row < m; row++) {
3         let index = col * m + row;
4         if (config[index] !== 0) {
5             return { value: config[index], index: index };
6         }
7     }
8     return null;
9 }

```

3.3.6 Full Column Verification

This function checks if a column is full by verifying that there are no zero values (empty spaces) in the column.

```

1 function isColumnFull(config, col) {
2     for (let row = 0; row < m; row++) {
3         let index = col * m + row;
4         if (config[index] === 0) {
5             return false;
6         }
7     }
8     return true;
9 }

```

3.3.7 Container Movement

This function simulates the movement of a container from a source column to a target column. It performs the following steps:

- Identifies the top container in the source column.
- Finds the first available space (a zero value) in the target column, starting from the bottom.
- Places the container in the target column if a valid space is found.

- Updates the source column by removing the container.

If no valid move is possible (e.g., the source column is empty or the target column is full), the function returns the original configuration unchanged.

```

1 function moveContainer(config, fromCol, toCol) {
2     const top = getTopContainer(config, fromCol);
3     if (!top) return config.slice();
4
5     let newConfig = config.slice();
6
7     // Remove container from source column
8     newConfig[top.index] = 0;
9
10    // Find position in target column
11    let destIndex = toCol * m + m - 1;
12    while (destIndex >= toCol * m && newConfig[destIndex] !== 0) {
13        destIndex--;
14    }
15
16    if (destIndex >= toCol * m) {
17        newConfig[destIndex] = top.value;
18    }
19
20    return newConfig;
21 }

```

3.3.8 Optimal Path Reconstruction

Once a sorted configuration is found, the algorithm reconstructs the path to it by tracing back through the *parent map*, a data structure that stores the relationships between configurations. Each key in the parent map is a configuration, and its value is the preceding configuration that led to it during exploration (previously mentioned in 3.3.1).

The process works as follows:

- Starting from the final (sorted) configuration, follow the parent references stored in the parent map back to the initial configuration.
- Record each configuration encountered along the way to form the sequence of moves.
- Reverse the sequence to present the path from the initial configuration to the sorted configuration in the correct order.

This path represents the **optimal solution**, guaranteed to be the shortest in terms of moves due to the use of Breadth-First Search (BFS). The length of the path minus one corresponds to the minimum number of moves required to reach the sorted configuration.

For example, given a parent map:

$$\text{ParentMap} = \text{Sorted} \rightarrow C2, C2 \rightarrow C1, C1 \rightarrow \text{Initial} \quad (2)$$

The path is reconstructed as:

Initial $\rightarrow$ C1 $\rightarrow$ C2 $\rightarrow$ Sorted.

This ensures that the algorithm not only finds a sorted configuration but also provides a complete sequence of moves to achieve it.

The path reconstruction function is implemented as:

```
1 function reconstructPath(config) {  
2   let path = [];  
3   let currentConfigStr = JSON.stringify(config);  
4   while (currentConfigStr !== null) {  
5     const currentConfig = JSON.parse(currentConfigStr);  
6     path.unshift(currentConfig);  
7     currentConfigStr = parentMap.get(currentConfigStr);  
8   }  
9   return path;  
10 }
```

3.3.9 JavaScript Code's Performance

In this subsection, we test the code's performance based on a given number of columns, rows, empty spaces, and priority numbers. We then measure the output of the code, which is the number of moves and the time taken to find the solution. Doing this will give us a strong understanding of the running time of the code and demonstrate its ability to always find the optimal solution.

It is important to mention that all tests were performed on the Lenovo Legion 5 Pro (CPU: Ryzen 7 7745HX; GPU: RTX 4070 Mobile), and the data provided can vary proportionally depending on the device.

All the tables consist of the following elements:

- **Initial configuration:** A randomly chosen configuration that has been tested.
- **Number of empty spaces:** The number of empty cells in the grid that can be used for container relocation.
- **Sorted configuration:** The sorted version of the initial configuration.
- **Number of moves:** The number of moves required to reach the sorted version of the initial configuration.
- **Time:** The time required to reach the sorted version of the initial configuration.

NB! Both the initial and sorted configurations will be displayed in the form of an array, as they are shown in the JavaScript code. In order to “read” and understand a configuration, the reader should pay attention to the number of rows and columns specified in a table. For instance, the configuration [6, 5, 2, 0, 0, 3, 0, 1, 4] (3 rows/3 columns) represents 3 stacks of containers:

- The first stack has 3 containers, where the container with priority number 6 is at the top of the stack and the container with priority number 2 is at the bottom of the stack respectively.
- The second stack has 1 container with priority number 3 and two empty spaces designated up with priority number 0.
- The third stack has 2 containers, where the container with priority number 1 is on the top of the stack and the container with priority number 4 is at the bottom of the stack respectively.

The number and ratio of rows and columns differ in each table. Half of the configurations in a table have empty spaces equal to the number of columns/rows (depending on which is larger), while the other half have fewer empty spaces than the number of columns/rows, respectively.

Unfortunately, some configurations take hours to solve using JavaScript code. Therefore, if it takes more than 1 hour to solve a configuration, we will use the C code implementation instead, as it is faster and finds the same sorted configurations as the JavaScript code implementation.

Ten of the most relevant configurations for each table are displayed in the main paper, while the additional data can be found in the appendices (Section B.3).

3 rows and 3 columns configuration instances				
Initial Configuration	No. of Empty Spaces	Sorted Configuration	No. of Moves	Time (s)
3, 5, 1, 4, 6, 2, 0, 0, 0	3	0, 0, 1, 0, 5, 6, 2, 3, 4	6	0.001
2, 6, 3, 0, 0, 4, 0, 5, 1	3	3, 5, 6, 0, 2, 4, 0, 0, 1	6	0.001
0, 0, 3, 0, 2, 1, 5, 6, 4	3	0, 0, 3, 0, 5, 6, 1, 2, 4	6	0.001
0, 3, 1, 5, 6, 4, 0, 0, 2	3	0, 5, 6, 1, 3, 4, 0, 0, 2	6	0.002
1, 6, 4, 0, 5, 2, 0, 0, 3	3	4, 5, 6, 0, 0, 2, 0, 1, 3	6	0.001
3, 5, 1, 4, 6, 2, 0, 0, 7	2	No sorted configuration	0	0.001
2, 6, 3, 0, 7, 4, 0, 5, 1	2	No sorted configuration	0	0.001
0, 7, 3, 0, 2, 1, 5, 6, 4	2	No sorted configuration	0	0.001
7, 3, 1, 5, 6, 4, 0, 0, 2	2	No sorted configuration	0	0.001
2, 5, 3, 7, 4, 1, 0, 0, 6	2	No sorted configuration	0	0.001

Table 1: The grid consists of 3 columns and 3 rows. The first five configurations have a number of empty spaces equal to the number of columns and rows, while the last five configurations have a number of empty spaces less than the number of columns and rows.

3 rows and 4 columns configuration instances				
Initial Configuration	No. of Empty Spaces	Sorted Configuration	No. of Moves	Time (s)
4, 3, 8, 0, 0, 2, 0, 0, 7, 5, 6, 1	4	2, 3, 8, 0, 0, 6, 4, 5, 7, 0, 0, 1	4	0.005
0, 0, 4, 0, 2, 8, 5, 3, 1, 0, 7, 6	4	0, 2, 4, 5, 7, 8, 0, 0, 1, 0, 3, 6	4	0.003
3, 8, 5, 4, 2, 6, 0, 0, 7, 0, 0, 1	4	0, 4, 5, 0, 2, 6, 1, 3, 7, 0, 0, 8	4	0.003
2, 8, 4, 0, 3, 5, 0, 1, 7, 0, 0, 6	4	0, 0, 8, 2, 3, 5, 0, 1, 7, 0, 4, 6	4	0.002
0, 6, 4, 0, 0, 5, 2, 7, 3, 0, 1, 8	4	0, 0, 4, 1, 2, 5, 0, 0, 3, 6, 7, 8	4	0.005
4, 3, 8, 0, 9, 2, 0, 0, 7, 5, 6, 1	3	2, 3, 8, 0, 6, 9, 4, 5, 7, 0, 0, 1	6	0.015
0, 9, 4, 0, 2, 8, 5, 3, 1, 0, 7, 6	3	4, 7, 9, 0, 2, 8, 0, 0, 1, 3, 5, 6	7	0.023
3, 8, 5, 4, 2, 6, 0, 9, 7, 0, 0, 1	3	0, 0, 5, 1, 2, 6, 3, 4, 7, 0, 8, 9	7	0.025
2, 8, 4, 9, 3, 5, 0, 1, 7, 0, 0, 6	3	0, 0, 4, 2, 3, 5, 0, 1, 7, 6, 8, 9	5	0.009
9, 6, 4, 0, 0, 5, 2, 7, 3, 0, 1, 8	3	0, 0, 4, 1, 2, 5, 0, 6, 9, 3, 7, 8	6	0.015

Table 2: The grid consists of 4 columns and 3 rows. The first five configurations have a number of empty spaces equal to the number of columns, while the last five configurations have a number of empty spaces less than the number of columns.

4 rows and 3 columns configuration instances				
Initial Configuration	No. of Empty Spaces	Sorted Configuration	No. of Moves	Time (s)
5, 3, 7, 8, 0, 4, 6, 1, 0, 0, 0, 2	4	1, 3, 7, 8, 0, 4, 5, 6, 0, 0, 0, 2	7	0.005
0, 5, 6, 2, 0, 1, 4, 3, 0, 0, 7, 8	4	0, 1, 2, 3, 0, 0, 0, 3, 4, 6, 7, 8	7	0.004
1, 7, 5, 8, 0, 0, 0, 4, 0, 6, 3, 2	4	2, 3, 5, 8, 0, 0, 1, 4, 0, 0, 6, 7	7	0.004
0, 1, 6, 5, 8, 4, 2, 3, 0, 0, 0, 7	4	0, 0, 1, 5, 0, 0, 2, 3, 4, 6, 7, 8	7	0.006
0, 5, 6, 7, 0, 2, 8, 1, 0, 0, 4, 3	4	1, 2, 6, 7, 0, 4, 5, 8, 0, 0, 0, 3	7	0.004
5, 3, 7, 8, 9, 4, 6, 1, 0, 0, 0, 2	3	No sorted configuration	0	0
9, 5, 6, 2, 0, 1, 4, 3, 0, 0, 7, 8	3	No sorted configuration	0	0
1, 7, 5, 8, 0, 0, 9, 4, 0, 6, 3, 2	3	No sorted configuration	0	0
9, 1, 6, 5, 8, 4, 2, 3, 0, 0, 0, 7	3	No sorted configuration	0	0
9, 5, 6, 7, 0, 2, 8, 1, 0, 0, 4, 3	3	No sorted configuration	0	0

Table 3: The grid consists of 3 columns and 4 rows. The first five configurations have a number of empty spaces equal to the number of rows, while the last five configurations have a number of empty spaces less than the number of rows.

4 rows and 4 columns configuration instances				
Initial Configuration	No. of Empty Spaces	Sorted Configuration	No. of Moves	Time (s)
2, 4, 8, 10, 0, 6, 1, 11, 0, 0, 3, 12, 0, 5, 9, 7	4	2, 4, 8, 10, 0, 0, 1, 11, 0, 6, 9, 12, 0, 3, 5, 7	6	0.080
10, 6, 8, 12, 3, 4, 2, 11, 0, 9, 5, 7, 0, 0, 0, 1	4	1, 6, 8, 12, 0, 0, 2, 11, 0, 3, 5, 7, 0, 4, 9, 10	6	0.022
7, 8, 1, 10, 11, 3, 4, 5, 2, 9, 6, 12, 0, 0, 0, 0	4	0, 0, 1, 10, 2, 3, 4, 5, 0, 0, 6, 12, 7, 8, 9, 11	6	88.030
6, 11, 3, 5, 2, 10, 4, 7, 0, 8, 1, 12, 0, 0, 0, 9	4	0, 2, 3, 5, 0, 0, 4, 7, 0, 10, 11, 12, 1, 6, 8, 9	6	0.043
0, 10, 5, 11, 6, 8, 9, 12, 1, 4, 2, 3, 0, 0, 0, 7	4	0, 1, 10, 11, 4, 8, 9, 12, 0, 0, 2, 3, 0, 5, 6, 7	6	0.036
2, 4, 8, 10, 13, 6, 1, 11, 0, 0, 3, 12, 0, 5, 9, 7	3	No sorted configuration	0	Greater than 3600
8, 5, 1, 2, 13, 9, 12, 10, 0, 11, 3, 6, 0, 0, 4, 7	3	No sorted configuration	0	Greater than 3600
6, 11, 3, 5, 2, 10, 4, 7, 13, 8, 1, 12, 0, 0, 0, 9	3	No sorted configuration	0	Greater than 3600
13, 10, 5, 11, 6, 8, 9, 12, 1, 4, 2, 3, 0, 0, 0, 7	3	No sorted configuration	0	Greater than 3600
7, 8, 1, 10, 11, 3, 4, 5, 2, 9, 6, 12, 0, 0, 0, 13	3	0, 0, 1, 10, 2, 3, 4, 5, 0, 8, 9, 12, 6, 7, 11, 13	7	0.029

Table 4: The grid consists of 4 columns and 4 rows. The first five configurations have a number of empty spaces equal to the number of columns and rows, while the last five configurations have a number of empty spaces less than the number of columns and rows.

3.4 Discussion and Improvements

Based on test results, the algorithm consistently finds the optimal solution with the minimum number of moves, regardless of the configuration. This is because BFS explores nodes level by level, maintaining distances from previous configurations, and ensuring the shortest path is always found. Therefore, we decided to use BFS instead of DFS. However, in both approaches, the algorithm’s running time increases exponentially. Beyond 4 columns and 4 rows (as shown by the results in Table 9), the running time becomes impractical due to the BRP being both an *NP*-hard and *NP*-complete problem, where the output size can grow extremely large (see 1.2.4).

In addition, the data shows that the more misplaced (unsorted) containers there are, the more moves and time it takes to find a solution. As mentioned in Section 2.3, empty space in a bay significantly impacts the number of moves required and whether a solution is achievable, which is prominently shown in Tables 1, 2, 3 and 4: if the available space is greater than or equal to the number of columns/rows (depending on which value is larger), a solution is guaranteed; however, if it is less, a solution is not always guaranteed respectively. Notably, the running time varies depending on the number of rows and columns, suggesting the possibility of an optimal grid configuration (the number of rows, columns, and empty space allows us to solve the problem). When we compare the time and number of moves from Table 1 with Table 4 we can see they increase faster when we increase the number of rows rather than the number of columns.

Determining the exact running time and optimal grid, however, remains inconclusive due to limited data and various variables, which require a more comprehensive analysis to model growth and running time accurately. In a nutshell, we consider that additional data combined with the configuration formula could provide better insights into the growth patterns [19].

4 Conclusion

To address our research question, “How can we develop an efficient algorithm for the Pre-Marshalling problem that would decrease economic losses?”, we implemented a BFS algorithm. While not perfect, this algorithm consistently finds the optimal solution for any configuration due to its traversal methods. However, results show that the running time increases exponentially, and beyond 4 columns and 4 rows, it becomes impractically large to measure. This prolonged computation time leaves containers unsorted for too long, meaning the algorithm fails to effectively reduce economic losses in intercontinental trade effectively.

The data indicates that running time varies depending on the available space, as well as the number of columns and rows. However, the growth rate remains inconclusive due to the limited dataset, making it impossible to derive a function at this stage. Additionally, mathematical proof is necessary to confirm the existence of an optimal number of rows and columns that, in its turn, will allow us to introduce additional heuristic rules that reduce the generation of redundant nodes in the graph and, respectively, the complexity of the code (as demonstrated in [15] and [18]).

For future research, we propose examining formula (1) in greater depth to find a lower bound for the number of moves. This could assist us in introduction of the aforementioned rules and heuristics that can optimize the BFS.

References

- [1] Michael R. Garey and David S. Johnson. *Computers and intractability*. eng. 1. New York: Freeman, 1999. ISBN: 0716710455.
- [2] Bart W Wiegman, Pieter Rietveld, and Peter Nijkamp. “Container terminal services and quality”. In: (2001).
- [3] Thomas H Cormen et al. *Introduction to Algorithms, Third Edition*. eng. 3rd ed. Cambridge: MIT Press, 2009. ISBN: 9780262033848.
- [4] Fedor V. Fomin and Dieter. Kratsch. *Exact exponential algorithms*. eng. Texts in theoretical computer science. Berlin: Springer, 2010. ISBN: 9783642165320.
- [5] Marco Caserta, Silvia Schwarze, and Stefan Voß. “Container rehandling at maritime container terminals”. In: *Handbook of terminal planning* (2011), pp. 247–269. DOI: https://doi.org/10.1007/978-1-4419-8408-1_13.
- [6] Robert Schaefer. “P, NP, and NP-Completeness: the basics of computational complexity by Oded Goldreich”. eng. In: *Software engineering notes* 36.1 (2011), pp. 37–38. ISSN: 0163-5948.
- [7] Andreas Bortfeldt and Florian Forster. “A tree search procedure for the container pre-marshalling problem”. In: *European Journal of Operational Research* 217.3 (2012), pp. 531–540. DOI: <https://doi.org/10.1016/j.ejor.2011.10.005>.
- [8] Marco Caserta, Silvia Schwarze, and Stefan Voß. “A mathematical formulation and complexity considerations for the blocks relocation problem”. In: *European Journal of Operational Research* 219.1 (2012), pp. 96–104. ISSN: 0377-2217. DOI: <https://doi.org/10.1016/j.ejor.2011.12.039>.
- [9] Marco Caserta, Silvia Schwarze, and Stefan Voß. “A mathematical formulation and complexity considerations for the blocks relocation problem”. In: *European Journal of Operational Research* 219.1 (2012), pp. 96–104. DOI: <https://doi.org/10.1016/j.ejor.2011.12.039>.
- [10] Christopher Expósito-Izquierdo, Belén Melián-Batista, and Marcos Moreno-Vega. “Pre-marshalling problem: Heuristic solution method and instances generator”. In: *Expert Systems with Applications* 39.9 (2012), pp. 8337–8349. DOI: <https://doi.org/10.1016/j.eswa.2012.01.187>.
- [11] Wenbin Zhu et al. “Iterative deepening A* algorithms for the container relocation problem”. In: *IEEE Transactions on Automation Science and Engineering* 9.4 (2012), pp. 710–722. DOI: <https://doi.org/10.1109/TASE.2012.2198642>.
- [12] Bo Jin, Wenbin Zhu, and Andrew Lim. “Solving the container relocation problem by an improved greedy look-ahead heuristic”. In: *European Journal of Operational Research* 240.3 (2015), pp. 837–847. DOI: <https://doi.org/10.1016/j.ejor.2014.07.038>.
- [13] Ning Wang, Bo Jin, and Andrew Lim. “Target-guided algorithms for the container pre-marshalling problem”. In: *Omega* 53 (2015), pp. 67–77. DOI: <https://doi.org/10.1016/j.omega.2014.12.002>.
- [14] Kevin Tierney, Dario Pacino, and Stefan Voß. “Solving the pre-marshalling problem to optimality with A* and IDA”. In: *Flexible Services and Manufacturing Journal* 29 (2017), pp. 223–259. DOI: <https://doi.org/10.1007/s10696-016-9246-6>.
- [15] Shunji Tanaka and Fumitaka Mizuno. “An exact algorithm for the unrestricted block relocation problem”. In: *Computers & Operations Research* 95 (2018), pp. 12–31. DOI: <https://doi.org/10.1016/j.cor.2018.02.019>.

- [16] Raka Jovanovic et al. “A GRASP approach for solving the Blocks Relocation Problem with Stowage Plan”. In: *Flexible Services and Manufacturing Journal* 31 (2019), pp. 702–729. DOI: <https://doi.org/10.1007/s10696-018-9320-3>.
- [17] Surabhi Sharma Sharma Surabh. *NTA UGC net / jrf / set paper-2 computer science & applications*. Arihant Publications India Ltd., 2019.
- [18] Shunji Tanaka and Stefan Voß. “An exact algorithm for the block relocation problem with a stowage plan”. In: *European Journal of Operational Research* 279.3 (2019), pp. 767–781. DOI: <https://doi.org/10.1016/j.ejor.2019.06.014>.
- [19] Stephan Miller. *What is the Fastest Programming Language?* Blog post on codecademy. 2022. URL: <https://www.codecademy.com/resources/blog/what-is-the-fastest-programming-language/#:~:text=The%20key%20feature%20of%20a,talks%20directly%20to%20the%20hardware>. (visited on 12/08/2024).
- [20] Everton Gomedé. *The Famous NP Problems*. Blog post on Medium. 2023. URL: <https://ai.plainenglish.io/the-famous-np-problems-42d3aef5069d> (visited on 11/28/2024).
- [21] Alf Kimms and Fabian Wilschewski. “A new modeling approach for the unrestricted block relocation problem”. In: *OR Spectrum* 45.4 (2023), pp. 1071–1111. DOI: <https://doi.org/10.1007/s00291-023-00728-w>.
- [22] Hilmi Amin Mohammed. “Ship emissions reduction via slow steaming without disrupting the logistical supply chain: A case study of the port of Felixstowe”. In: *Journal of International Maritime Safety, Environmental Affairs, and Shipping* 7.4 (2023), p. 2280416. DOI: <https://doi.org/10.1080/25725084.2023.2280416>.
- [23] Equipo de GeeksforGeeks. “Introduction to Map - Data Structure and Algorithm Tutorials”. In: *GeeksforGeeks* (2024). URL: <https://www.geeksforgeeks.org/introduction-to-map-data-structure/>.
- [24] Equipo de GeeksforGeeks. “Introduction to Tree Data Structure”. In: *GeeksforGeeks* (2024). URL: <https://www.geeksforgeeks.org/introduction-to-tree-data-structure/>.
- [25] OpenAI. *ChatGPT (gpt-4o)*. 2024. URL: <https://chat.openai.com>.
- [26] OpenAI. *OpenAI o1 (o1-preview)*. 2024. URL: <https://chat.openai.com>.
- [27] Siim Sild. *Supply Chain Issues: Port Congestion*. Fractory. 2024. URL: <https://fractory.com/port-congestion-explained/#:~:text=Financial%20burdens%2C%20such%20as%20demurrage,within%20the%20allotted%20free%20days>. (visited on 12/06/2024).

5 Glossary

Most of the symbols and variables will be used mainly for the demonstration shown in appendix.

Symbol	Definition
m	Number of rows in the bay.
n	Number of columns in the bay.
z	Number of destinations or priorities.
j_i	Number of containers assigned to the i -th destination or priority, for $i = 1, 2, \dots, z$.
c	Total number of containers, calculated as $c = \sum_{i=1}^z j_i$.
E	Number of empty positions in the bay, calculated as $E = mn - c$.
$\zeta(m, n, c, \{j_i\}_{i=1}^z)$	Function returning the total number of valid configurations based on $m, n, c, \{j_i\}_{i=1}^z$.
S	Number of valid combinations where the empty spaces are correctly placed.
S'	Number of combinations where the empty slots are incorrectly placed, defined as $S' = T - S$.
T	Total number of combinations of container placements, calculated as $T = \binom{mn}{c}$.
W	Number of invalid configurations due to gravity constraint violations (containers "floating").
A_q	The set of combinations where the q -th column has at least $m + 1$ empty spaces (exceeding the possible limit of spaces in a column).
η	The set of combinations where at least one column has more than m empty spaces.
x	Column index, $x \in \{1, 2, \dots, n\}$.
y	Row index, $y \in \{1, 2, \dots, m\}$.
(x, y)	A position in the bay.
x_o	Number of empty positions in the o -th column, $x_o \in \mathbb{N} \cup \{0\}$.
$\mathcal{C}$	The set of all containers, with $c = \mathcal{C} $.
$p(C)$	Priority of container C , $p(C) \in \{1, 2, \dots, z\}$.
$\mathcal{P}$	The set of positions in the bay, $\mathcal{P} = \{(x, y) \mid 1 \leq x \leq n, 1 \leq y \leq m, x, y \in \mathbb{N}\}$.
e_x	Number of empty positions in column x , $e_x \in \mathbb{N} \cup \{0\}$.
s_x	Number of occupied positions in column x , $s_x = m - e_x$.
f	Function assigning containers to positions, $f : \mathcal{C} \rightarrow \mathcal{P}$.
$\mathcal{G}_{\text{valid}}$	Set of valid configurations (injective, complete, satisfying gravity constraint).
$\mathcal{H}_i$	A subset of $\mathcal{G}_{\text{valid}}$ where all elements of the subset can be transformed to another element of the subset in a finite number of transformations that follow the constraints.
$\mathcal{G}_{\text{total}}$	Set of all injective and complete functions from $\mathcal{C}$ to $\mathcal{P}$.
γ	Total number of injective and complete functions from $\mathcal{C}$ to $\mathcal{P}$, $\gamma = \mathcal{G}_{\text{total}} $.
λ	Number of invalid configurations due to gravity constraint violations, $\lambda = \gamma - \zeta$.
$\mathcal{E}$	Vector $(e_1, e_2, \dots, e_n)$ representing the number of empty positions in each column.
A_μ	The set of configurations where $e_\mu \geq m + 1$.

Symbol	Definition
E'	Adjusted number of empty spaces, $E' = E - k(m + 1)$.
k	Number of columns exceeding the empty space limit in the inclusion-exclusion principle.
$\prod_{i=1}^z j_i!$	Product of the factorials of j_i , used in the multinomial coefficient.
$(\mathbb{N} \cup \{0\})^n$	The set of n -tuples of non-negative integers.
$\Gamma(x)$	Gamma function, an extension of the factorial function to complex numbers.
$\sim$	Equivalence relation indicating that containers with the same priority are indistinguishable.
S_x	Set of occupied positions in column x , $S_x = \{y \in \{1, 2, \dots, m\} \mid (x, y) \in f(\mathcal{C})\}$.
$f(C)$	Position assigned to container C by function f .
C	A container in $\mathcal{C}$.
$\binom{a}{b}$	Binomial coefficient, equal to zero when $a \in \mathbb{R}^-$ or $a < b$.

A Initial Configurations Formula Explanation and Proof

The following proof and the derivation of the formula presented in this section were developed and rigorously verified by Jiaxu Yang.

The nomenclature that follows will be used to denote the various problem scenarios:

$$mRnC(j_1, j_2, j_3, \dots, j_z) : m, n, z, j_i \in \mathbb{N}, \forall i \in \mathbb{N}$$

In this scenario, m stands for the number of rows, while n refers to the number of columns, z is the number of destinations or priorities, and (j_i) indicates the number of containers assigned to the i -th destination or priority. For example, consider a bay with 2 rows, 3 columns, and 4 containers distributed across different priorities: 1 container for the first destination, 1 for the second, and 2 for the third. This configuration can be written as: $2R3C(1, 1, 2)$. One possibility would be the one shown in Figure 10.

1		
2	3	3

Figure 10: Example of a $2R3C(1, 1, 2)$ configuration.

This notation does not specify the exact arrangement of the containers in the bay.

To put this into perspective, let's start by analyzing the following problem, $2R2C(1, 1, 1)$. By manually exploring with pen and paper, we can find that there are 12 distinct configurations in total, and we will denote this number as ζ . However, for larger values of m , n , c and z , it is not feasible to manually find all configurations, so we begin by analyzing this mathematically.

1		1		2		2		3		3	
2	3	3	2	1	3	3	1	1	2	2	1
	1		1		2		2		3		3
2	3	3	2	1	3	3	1	1	2	2	1

Figure 11: All possible $2R2C(1, 1, 1)$ configurations.

Using basic combinatorial principles and permutation theory, we can see that, for any container configuration, the following inequality always holds:

$$\zeta(m, n, c) \leq \binom{mn}{c} c!$$

Where c is the numbers of containers an we can also define ζ as a function that depends on all the variables mentioned above.

The binomial coefficient formula tells us how many ways we can place c containers into mn available slots, while the permutation formula gives the number of ways the containers can be rearranged. This formula is not always an equality because some configurations are invalid, certain configurations may violate the gravity constraint, causing containers to “float”.

Applying this formula to our $2R2C(1, 1, 1)$ example, we get a result of 24, confirming the inequality. We also notice that the actual number of valid configurations is exactly half, and this is no coincidence. Since there is one empty slot (due to $mn = 4$ and $c = 3$), this empty space must always be in one of the two top positions and never in the bottom two, reducing the total count by half.

Now, let's consider a more complex version of the problem. In this scenario, the containers are not uniquely assigned to each destination. The problem $2R2C(2, 1)$ has only two destinations, but two containers are bound for the same destination. Logically, we expect fewer combinations in this case because two containers share the same priority, making them interchangeable. Thus, we modify the permutation formula to use the multinomial coefficient:

$$\zeta(m, n, c, \{j_i\}_{i=1}^z) \leq \binom{mn}{c} \frac{c!}{j_1! j_2! j_3! \dots j_z!} = \binom{mn}{c} \frac{c!}{\prod_{i=1}^z j_i!}$$

However, this formula is still incomplete because it represents an inequality. To convert it into an equality, we need to subtract W , which accounts for all the invalid configurations where containers "float" or, in more formal terms, where the empty slots are incorrectly placed.

$$\zeta(m, n, c, \{j_i\}_{i=1}^z) = \binom{mn}{c} \frac{c!}{\prod_{i=1}^z j_i!} - W$$

By intuition, we define W as the number of permutations of the containers multiplied by the configurations where the empty space is incorrectly placed (S'), in other words:

$$W = \frac{c!}{\prod_{i=1}^z j_i!} \cdot S'$$

Where $S' = T - S$, T is the total number of combinations of empty spaces and S is the combinations where the empty space is correctly placed.

The Stars and Bars theorem says that the number of equations where all the x_o are non-negative integers ($x_o \in \mathbb{N} \cup \{0\}$), such that $o \in \mathbb{N}$, $o \leq n$:

$$x_1 + x_2 + x_3 + \dots + x_n = E$$

Where $E = mn - c$, meaning that E is the quantity of empty spaces; in this case, each x_o represents a column and we can also express it as:

$$\sum_{o=1}^n x_o = E$$

So according to the theorem, the number of valid equations would be given by the following equation (It should be noted that the Stars and Bars theorem does not have an upper restriction, so for now we will refer to the valid combinations as S instead of using the theorem since later we will provide a more generalized definition for S):

$$mn - c \leq m \implies S = \binom{E + n - 1}{n - 1}$$

One problem of the Stars and Bars theorem is that it tells you the number of possible integer solutions without an upper restriction, so this formula ceases to work if $mn - c > m$, but for now, we will continue with the reasoning to see the development of the formula, and later we can modify the Stars and Bars theorem to eliminate these invalid solutions.

So the equation is:

$$\binom{mn}{c} \frac{c!}{\prod_{i=1}^z j_i!} - \frac{c!}{\prod_{i=1}^z j_i!} \left(\binom{mn}{c} - S \right)$$

However, upon closer inspection, we can observe that the formula is also equivalent to the product of the permutations of the containers and the combinations of well-placed empty spaces. Additionally, this conclusion can be reached simply by further simplifying the previous expression, which results in:

$$\frac{c!}{\prod_{i=1}^z j_i!} \cdot S$$

Now, we will provide a more sophisticated definition for S based on the Stars and Bars theorem so that it works not only when $mn - c \leq m$.

One way to solve this problem is by eliminating all those combinations where the Stars and Bars theorem exceeds the height limit; all these combinations will form a set η .

$$S = \binom{E + n - 1}{n - 1} - \eta$$

If we assume that we have 3 columns, in this case, η will be the same as the number of elements in the union of the sets A_1 , A_2 , A_3 , where A_q is the number of combinations in which the q -th column has at least $m + 1$ empty spaces (i.e., exceeding the possible limit of spaces in a column).

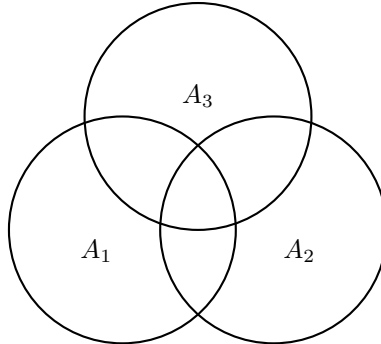


Figure 12: Union of 3 sets.

Thus,

$$\begin{aligned}
\eta &= |A_1 \cup A_2 \cup A_3| \\
&= |A_1| + |A_2| + |A_3| \\
&\quad - |A_1 \cap A_2| - |A_1 \cap A_3| - |A_2 \cap A_3| \\
&\quad + |A_1 \cap A_2 \cap A_3|
\end{aligned}$$

Fortunately, there exists a principle known as the inclusion-exclusion principle that provides us with a general formula for calculating the cardinality of the union of n sets. What we want to calculate is the cardinality of the union of all sets A_q :

$$\eta = \left| \bigcup_{q=1}^n A_q \right| = \sum_{\emptyset \neq P \subseteq \{1,2,\dots,n\}} (-1)^{|P|+1} \left| \bigcap_{i \in P} A_i \right|$$

Otra manera de escribir el principio de inclusion y exclusion es la siguiente:

$$\sum_{k=1}^n (-1)^{k+1} \sum_{1 \leq i_1 < \dots < i_k \leq n} |A_{i_1} \cap \dots \cap A_{i_k}|$$

Now we can apply the principle in our case, and we would obtain the following result:

$$S = \binom{E+n-1}{n-1} - \sum_{k=1}^n (-1)^{k-1} \binom{n}{k} \binom{E-k(m+1)+n-1}{n-1}$$

And in a simplified manner, it could be expressed as:

$$S = \sum_{k=0}^n (-1)^k \binom{n}{k} \binom{E-k(m+1)+n-1}{n-1}$$

Finally, we conclude that

$$\zeta(m, n, c, \{j_i\}_{i=1}^z) = \frac{c!}{\prod_{i=1}^z j_i!} \sum_{k=0}^n (-1)^k \binom{n}{k} \binom{mn-c-k(m+1)+n-1}{n-1} \quad (3)$$

Now, additionally it is important to take into account that due to the use of several factorials, summations, and products this formula can grow even more than exponential, because $\lim_{x \rightarrow \infty} \frac{x!}{a^x} = \infty \mid \forall a \in \mathbb{R}^+$, in complementary we can consider that some combinations are homologous with others since the way of solving would be identical and these are exactly the same containers only that in another column, reason why the variant of this formula to calculate the number of configurations if we ignore that the homologous columns would be equal to the previously mentioned formula but divided between $n!$ as it is obvious.

As a final addition, when entering the formula, an error can occur, as often calculators and computers do not define the binomial coefficient when the upper index is a negative number.

If we calculate it using factorials, factorials of negative integers are not defined (even if we use the Gamma function $\Gamma(x)$). In our case, we will assume that if the upper index is a negative number or smaller than the lower one, the binomial coefficient is equal to 0.

$$\binom{a}{b} = 0: a \in \mathbb{R}^- \vee a < b$$

A.1 Proof

Obviously the previous section cannot be considered as a formal proof, but rather an explanation of the methodology to reach the result with a more informative and easy to understand objective for everyone. In this section we will formally demonstrate the formula (1), if the reader is not familiar with the concepts such as set theory that will be mentioned next we recommend to skip this section since we consider that the explanation of the previous section is enough to understand the formula and that it is not the central topic of the investigation although it is trivial to justify with a mathematical and logical solidity the concepts of the investigation for the most rigorists.

Statement. Given $m, n, c, \{j_i\}_{i=1}^z$ and let:

$$\zeta = \left| \left\{ f: \mathcal{C} \rightarrow \mathcal{P} \left| \begin{array}{l} f \text{ is injective} \\ f \text{ is completeness} \\ \forall C \in \mathcal{C}: f(C) = (x, y) \wedge y > 1 \implies \exists C' \in \mathcal{C}: f(C') = (x, y) \end{array} \right. \right\} \right|$$

Then:

$$\zeta(m, n, c, \{j_i\}_{i=1}^z) = \frac{c!}{\prod_{i=1}^z j_i!} \sum_{k=0}^n (-1)^k \binom{n}{k} \binom{mn - c - k(m+1) + n - 1}{n - 1}$$

A.1.1 Definitions and Notation

Let $\mathcal{C}$ represent the collection of all containers and let $c = |\mathcal{C}|$ where $c \leq mn$. Each container $C \in \mathcal{C}$ possesses:

1. A priority $p(C) \in \{1, 2, 3, \dots, z\}$.
2. The characteristic that if two or more containers share the same priority, they are indistinguishable (denoted by the symbol $\sim$).

The number of containers for each priority is $i: 1 \leq i \leq z$ is j_i , thus: $\sum_{i=1}^z j_i = c$.

The bay is a grid $\mathcal{P}$ of size $n \times m$ with positions:

$$\mathcal{P} = \{(x, y) : x \leq n \wedge y \leq m \wedge x \in \mathbb{N} \wedge y \in \mathbb{N}\}$$

Let $E = mn - c$ denote the total number of empty positions in the bay. Let e_x be the number of empty positions in column x , so:

$$\sum_{x=1}^n e_x = E: e_x \in \mathbb{N} \cup \{0\} \wedge e_x \leq m$$

The number of occupied positions in column x is $s_x = m - e_x$.

A valid configuration is an assignment $f : \mathcal{C} \rightarrow \mathcal{P}$ that satisfies:

1. Injectivity: No two containers occupy the same position, meaning if $C_1 \neq C_2$, then $f(C_1) \neq f(C_2)$.
2. Gravity Constraint: If a container is placed at position (x, y) where $y > 1$, there must be a container at position $(x, y - 1)$. Formally, for all $C \in \mathcal{C}$, if $f(C) = (x, y)$ and $y > 1$, then there exists $C' \in \mathcal{C}$ such that $f(C') = (x, y - 1)$.
3. Completeness: All containers must be within the bay, meaning f is a total function.

We will define some sets that will be frequently used in the proof:

$$\mathcal{G}_{\text{valid}} = \left\{ f : \mathcal{C} \rightarrow \mathcal{P} \mid \begin{array}{l} f \text{ is injective} \\ f \text{ satisfies the gravity constraint} \\ f \text{ ensures completeness} \end{array} \right\}$$

Therefore, a valid configuration is an element belonging to this set, and $\zeta = |\mathcal{G}_{\text{valid}}|$.

$$\mathcal{G}_{\text{total}} = \left\{ f : \mathcal{C} \rightarrow \mathcal{P} \mid \begin{array}{l} f \text{ is injective} \\ f \text{ ensures completeness} \end{array} \right\}$$

A.1.2 Fundamental Principles

Theorem 1 (Stars and Bars Theorem). *The number of non-negative solutions to the equation:*

$$x_1 + x_2 + x_3 + \cdots + x_k = n$$

is:

$$\binom{n+k-1}{k-1} : n \in \mathbb{N} \cup \{0\} \wedge k \in \mathbb{N}$$

Theorem 2 (Inclusion-Exclusion Principle). *For finite sets $A_1, A_2, A_3, \dots, A_n$:*

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{k=1}^n (-1)^{k+1} \sum_{1 \leq i_1 < \cdots < i_k \leq n} |A_{i_1} \cap \cdots \cap A_{i_k}|$$

Theorem 3 (Binomial Coefficient). *The number of ways to choose k distinct elements from a set of n elements without regard to order is:*

$$\binom{n}{k} = \frac{n!}{k! (n-k)!}$$

If $k > n$ or $k < 0$, then $\binom{n}{k} = 0$, since there are no ways to choose k elements from a set of n elements, nor can you choose a negative number of elements.

Theorem 4 (Multinomial Coefficient). *If there are n elements to be divided into k groups of sizes $n_1, n_2, n_3, \dots, n_k$ such that $\sum_{i=1}^k n_i = n$, the number of ways to arrange these elements is:*

$$\frac{n!}{n_1!n_2!n_3!\dots n_k!} = \frac{n!}{\prod_{i=1}^k n_i!}$$

Theorem 5 (The Division Theorem). *For any integer a and any positive integer b , there exist unique integers q and r that satisfy the following equation:*

$$a = qb + r$$

Where $0 \leq r < b$.

Lemma 1. *Given $n, m, c, \{j_i\}_{i=1}^z$, there always exists at least one valid configuration.*

Proof. We will explicitly construct a valid configuration f_1 :

1. Arrange the containers in $\mathcal{C}$ in some fixed order: $C_1, C_2, C_3, \dots, C_c$.
2. For each container C_k where $1 \leq k \leq c$, define:

$$x = 1 + ((k-1) \bmod n), \quad y = 1 + \left\lfloor \frac{k-1}{n} \right\rfloor$$

Assign $f(C_k) = (x, y)$.

3. Since $c \leq mn$, we have:

$$y = 1 + \left\lfloor \frac{k-1}{n} \right\rfloor \leq 1 + \left\lfloor \frac{mn-1}{n} \right\rfloor = 1 + (m-1) = m$$

Therefore, $1 \leq y \leq m$.

By the definition of the modulo operation:

$$x_k = 1 + ((k-1) \bmod n).$$

We know that $(k-1) \bmod n \in \{0, 1, 2, \dots, n-1\}$, hence:

$$x_k \in \{1, 2, \dots, n\}.$$

This ensures that both x and y adhere to the established bounds.

4. Suppose that $f(C_{k_1}) = f(C_{k_2})$ for all $k_1, k_2 \in \{1, 2, 3, \dots, c\}$, then:

$$(x_{k_1}, y_{k_1}) = (x_{k_2}, y_{k_2})$$

This implies that:

$$1 + ((k_1-1) \bmod n) = 1 + ((k_2-1) \bmod n)$$

and also that:

$$1 + \left\lfloor \frac{k_1-1}{n} \right\rfloor = 1 + \left\lfloor \frac{k_2-1}{n} \right\rfloor$$

Simplifying, we obtain:

$$((k_1-1) \bmod n) = ((k_2-1) \bmod n)$$

and also:

$$\left\lfloor \frac{k_1 - 1}{n} \right\rfloor = \left\lfloor \frac{k_2 - 1}{n} \right\rfloor$$

Define:

$$\begin{aligned} r &= ((k_1 - 1) \bmod n) = ((k_2 - 1) \bmod n) \\ q &= \left\lfloor \frac{k_1 - 1}{n} \right\rfloor = \left\lfloor \frac{k_2 - 1}{n} \right\rfloor \end{aligned}$$

Then, we can express $k_1 - 1$ and $k_2 - 1$ in terms of q and r :

$$k_1 - 1 = qn + r$$

$$k_2 - 1 = qn + r$$

Using Theorem 5, we know both expressions are equal:

$$k_1 - 1 = k_2 - 1 \implies k_1 = k_2$$

Therefore, the function f_1 is injective.

5. The function is defined for each C_k and since $|\mathcal{C}| \leq |\mathcal{P}|$, the function is total and thus satisfies the completeness property.
6. For any container C_k with $y_k > 1$, consider the position $(x_k, y_k - 1)$.

Observe that:

$$y_k - 1 = \left(1 + \left\lfloor \frac{k - 1}{n} \right\rfloor \right) - 1 = \left\lfloor \frac{k - 1}{n} \right\rfloor$$

Since $y_k > 1$, it implies that $\left\lfloor \frac{k-1}{n} \right\rfloor \geq 1$, therefore, there exists $C_{k'}$ such that:

$$k' = k - n: k' \geq 1$$

Then:

$$f(C_{k'}) = \left(1 + ((k' - 1) \bmod n), 1 + \left\lfloor \frac{k' - 1}{n} \right\rfloor \right)$$

Substituting $k' = k - n$:

$$f(C_{k-n}) = \left(1 + ((k - n - 1) \bmod n), 1 + \left\lfloor \frac{k - n - 1}{n} \right\rfloor \right)$$

We observe that:

$$x_{k-n} = 1 + ((k - n - 1) \bmod n) = 1 + ((k - 1) \bmod n) = x_k,$$

and

$$y_{k-n} = 1 + \left\lfloor \frac{k - n - 1}{n} \right\rfloor = y_k - 1$$

Therefore, $f(C_{k-n}) = (x_k, y_k - 1)$, thereby satisfying the gravity constraint.

□

Lemma 2. Let $\gamma = |\mathcal{G}_{total}|$. Then, given $n, m, c, \{j_i\}_{i=1}^z$, it always holds that:

$$\gamma = \binom{mn}{c} \frac{c!}{\prod_{i=1}^z j_i!}$$

Proof. We define a finite number of subsets of $\mathcal{G}_{\text{total}}$ where each subset corresponds to a specific selection of c positions in the bay $\mathcal{P}$. Specifically, for each subset $S \subseteq \mathcal{P}$ with $|S| = c$, we define:

$$\mathcal{G}_S = \left\{ f \in \mathcal{G}_{\text{total}} \mid f(\mathcal{C}) = S \right\}$$

Now, consider two distinct subsets S_1 and S_2 such that $S_1 \neq S_2$. Suppose, for contradiction, that there exists a configuration f that belongs to both $\mathcal{G}_{S_1}$ and $\mathcal{G}_{S_2}$. Then by definition:

$$f(\mathcal{C}) = S_1$$

$$f(\mathcal{C}) = S_2$$

Since f is an injective function, this implies that $S_1 = S_2$, which contradicts our initial assumption. Therefore, all subsets $\mathcal{G}_S$ are mutually disjoint.

Consider any configuration $f \in \mathcal{G}_{\text{total}}$. By definition, f assigns the containers to a subset of c positions in $\mathcal{P}$. In other words, there exists a set $S \subseteq \mathcal{P}$ with $|S| = c$ such that $f \in \mathcal{G}_S$. Thus, each element of $\mathcal{G}_{\text{total}}$ belongs to at least one of the subsets $\mathcal{G}_S$. Hence, the union of all subsets $\mathcal{G}_S$ completely covers $\mathcal{G}_{\text{total}}$:

$$\mathcal{G}_{\text{total}} = \bigcup_{\substack{S \subseteq \mathcal{P} \\ |S|=c}} \mathcal{G}_S$$

To determine $|\mathcal{G}_S|$, we use Theorem 4, as the number of ways to assign the c containers to the c positions in S , considering that indistinguishability by priority is given by the multinomial coefficient:

$$|\mathcal{G}_S| = \frac{c!}{\prod_{i=1}^z j_i!}$$

Finally, let N be the number of possible subsets $S \subseteq \mathcal{P}$ with $|S| = c$, which is given by Theorem 3:

$$N = \binom{mn}{c}$$

Therefore, since the subsets $\mathcal{G}_S$ are disjoint and all have the same cardinality, we know that:

$$\gamma = N |\mathcal{G}_S|$$

Thus, we have shown that:

$$\gamma = \binom{mn}{c} \frac{c!}{\prod_{i=1}^z j_i!}$$

□

Lemma 3. Let $\zeta = |\mathcal{G}_{\text{valid}}|$ and $\gamma = |\mathcal{G}_{\text{total}}|$. Then, given $n, m, c, \{j_i\}_{i=1}^z$, it always holds that:

$$\zeta \leq \gamma$$

Proof. All elements of $\mathcal{G}_{\text{valid}}$ also satisfy the conditions of $\mathcal{G}_{\text{total}}$, and therefore:

$$\mathcal{G}_{\text{valid}} \subseteq \mathcal{G}_{\text{total}}$$

Consequently, the cardinality of $\mathcal{G}_{\text{valid}}$ cannot exceed the cardinality of $\mathcal{G}_{\text{total}}$, that is:

$$\zeta = |\mathcal{G}_{\text{valid}}| \leq |\mathcal{G}_{\text{total}}| = \gamma$$

This directly establishes that $\zeta \leq \gamma$. $\square$

Lemma 4. *For any $f \in \mathcal{G}_{\text{valid}}$ and for any column $x \in \{1, 2, 3, \dots, n\}$, there exists an integer $s_x \in \{1, 2, 3, \dots, m\}$ such that:*

$$f(\mathcal{C}) \cap (\{x\} \times \{1, 2, 3, \dots, m\}) = \{(x, y) \mid 1 \leq y \leq s_x\}$$

Proof. Let $f \in \mathcal{G}_{\text{valid}}$. For each column x , define:

$$S_x = \{y \in \{1, 2, 3, \dots, m\} \mid (x, y) \in f(\mathcal{C})\}$$

We aim to show that S_x is an initial segment of $\{1, 2, 3, \dots, m\}$, that is, there exists $s_x \in \{1, 2, 3, \dots, m\}$ such that:

$$S_x = \{1, 2, 3, \dots, s_x\}$$

We proceed by contradiction. Suppose that S_x is not an initial segment, meaning there exists a $y \in S_x$ such that $y > 1$ and $y - 1 \notin S_x$. This would violate the gravity constraint. Therefore, S_x must be an initial segment, implying that $\exists s_x$. $\square$

A.1.3 Logic

Proof. Given $n, m, c, \{j_i\}_{i=1}^z$, we know that there always exists at least one valid configuration by Lemma 1, and the total number of valid configurations must satisfy $\zeta \leq \gamma$ by Lemma 3.

$$1 \leq \zeta \leq \gamma$$

We know that both ζ and γ are cardinalities of sets, and therefore are natural numbers. Thus:

$$\begin{aligned} \zeta &\leq \gamma \\ \gamma - \zeta &\geq 0 \end{aligned}$$

Hence, we know that there exists a $\lambda \in \mathbb{N} \cup \{0\}$ that satisfies $\zeta = \gamma - \lambda$, in other words:

$$\exists \lambda : \lambda \in \mathbb{N} \cup \{0\} \wedge \zeta = \gamma - \lambda$$

By the definitions of ζ and γ , λ must be defined as:

$$\lambda = |\mathcal{G}_{\text{invalid}}| = \left| \left\{ f : \mathcal{C} \rightarrow \mathcal{P} \mid \begin{array}{l} f \text{ is injective} \\ f \text{ ensures completeness} \\ \exists C \in \mathcal{C} : f(C) = (x, y) \wedge y > 1 \wedge \nexists C' \in \mathcal{C} : f(C') = (x, y - 1) \end{array} \right\} \right|$$

Since the sum of the cardinalities that satisfy and violate the gravity constraint is equivalent to the cardinality of the set that does not account for the constraint, in other words, $\zeta + \lambda = \gamma$.

Both ζ and λ are sets of functions differentiated by the positions of the empty spaces. As established in Lemma 2, $\mathcal{G}_{\text{total}}$ can be partitioned into disjoint subsets $\mathcal{G}_S$, where each $\mathcal{G}_S$ corresponds to a specific selection of c positions in the bay $\mathcal{P}$. Formally:

$$\mathcal{G}_{\text{total}} = \bigcup_{\substack{S \subseteq \mathcal{P} \\ |S|=c}} \mathcal{G}_S,$$

where for each $S \subseteq \mathcal{P}$ with $|S| = c$:

$$\mathcal{G}_S = \left\{ f \in \mathcal{G}_{\text{total}} \mid f(\mathcal{C}) = S \right\}.$$

If any element of $\mathcal{G}_S$ satisfies the gravity constraint, then all elements of the same subset do as well, because the functions are bijective. Similarly, if any element of the subset violates the constraint, all elements will violate it. Knowing that the subsets $\mathcal{G}_S$ are all disjoint, we deduce that:

$$\zeta = A |\mathcal{G}_S|$$

$$\lambda = B |\mathcal{G}_S|$$

where A and B are the number of such subsets $\mathcal{G}_S$ in their respective categories. Since they are subsets of $\mathcal{G}_{\text{total}}$, we know $|\mathcal{G}_S|$, therefore:

$$\zeta = A \frac{c!}{\prod_{i=1}^z j_i!}$$

Lemma 4 tells us that if each column has s_x containers, there is only one unique way to occupy those spaces. Therefore, there exists only one unique way to leave spaces unoccupied, meaning A is the number of solutions to the following equation:

$$\sum_{x=1}^n e_x = E : e_x \in \mathbb{N} \cup \{0\} \wedge e_x \leq m$$

Each combination of e_x that satisfies the equation corresponds to a subset $\mathcal{G}_S$, thus it follows that:

$$A = \left| \left\{ \mathcal{E} = (e_1, e_2, e_3, \dots, e_n) \in (\mathbb{N} \cup \{0\})^n \mid \sum_{x=1}^n e_x = E \wedge e_x \in \mathbb{N} \cup \{0\} \right\} \right|$$

We know that Theorem 1 provides the number of solutions to these equations but without an upper limit. We define A_μ as:

$$A_\mu = \left| \left\{ \mathcal{E} = (e_1, e_2, e_3, \dots, e_n) \in (\mathbb{N} \cup \{0\})^n \mid \sum_{x=1}^n e_x = E \wedge e_x \in \mathbb{N} \cup \{0\} \wedge e_\mu \geq m+1 \right\} \right|$$

and also define A_κ for $\kappa \subseteq \{1, 2, 3, \dots, n\}$ as:

$$A_\kappa = \bigcap_{\mu \in \kappa} A_\mu$$

Knowing that:

$$A = \binom{mn - c + n - 1}{n - 1} - \left| \bigcup_{\mu=1}^n A_\mu \right|$$

We expand the expression using Theorem 2:

$$\left| \bigcup_{\mu=1}^n A_{\mu} \right| = \sum_{k=1}^n \left((-1)^{k+1} \sum_{\substack{\kappa \subseteq \{1,2,3,\dots,n\} \\ |\kappa|=k}} |A_{\kappa}| \right)$$

Now we define E' to adjust it to the inclusion-exclusion principle:

$$E' = E - k(m+1) = mn - c - k(m+1)$$

Thus:

$$|A_{\kappa}| = \binom{E' + n - 1}{n - 1} = \binom{mn - c - k(m+1) + n - 1}{n - 1}$$

Substituting the expression:

$$\left| \bigcap_{\mu=1}^n A_{\mu} \right| = \sum_{k=1}^n (-1)^{k+1} \binom{n}{k} \binom{mn - c - k(m+1) + n - 1}{n - 1}$$

Replacing the expression:

$$A = \binom{mn - c + n - 1}{n - 1} - \sum_{k=1}^n (-1)^{k+1} \binom{n}{k} \binom{mn - c - k(m+1) + n - 1}{n - 1}$$

Evidently, we can simplify the expression to:

$$A = \sum_{k=0}^n (-1)^k \binom{n}{k} \binom{mn - c - k(m+1) + n - 1}{n - 1}$$

Finally, we substitute A and express ζ as a function, demonstrating that:

$$\zeta(m, n, c, \{j_i\}_{i=1}^z) = \frac{c!}{\prod_{i=1}^z j_i!} \sum_{k=0}^n (-1)^k \binom{n}{k} \binom{mn - c - k(m+1) + n - 1}{n - 1}$$

□

B Code

B.1 BFS Algorithm in JavaScript

The code has been developed by Jiaxu Yang, Ilie Baciuc and Sabin Ghimire, with the assistance of AI tools such as OpenAI o1-preview [26][25] for certain code fragments.

```
1  const m = 4; // Number of rows
2  const n = 4; // Number of columns
3
4  const initial_config = [
5      0, 3, 1, 2,
6      5, 4, 3, 5,
7      0, 0, 5, 4,
8      0, 5, 4, 1,
9  ];
10
11 const visited = new Set();
12 const parentMap = new Map();
13 const queue = [];
14 let found = false;
15
16 // Formatted output of the configuration
17 function print_config(config) {
18     for (let i = 0; i < m; i++) {
19         let row = '';
20         for (let j = 0; j < n; j++) {
21             const value = config[j * m + i];
22             row += value.toString().padStart(3, ' ') + ' '; // Aligns to
23             width
24         }
25         console.log(row.trim());
26     }
27     console.log();
28 }
29
30 function generate_moves(config) {
31     let moves = [];
32     const columns = Array.from({ length: n }, (_, i) => i);
33
34     for (let fromCol of columns) {
35         const topContainer = get_top_container(config, fromCol);
36
37         if (topContainer !== null) {
38             for (let toCol of columns) {
39                 if (toCol !== fromCol) {
40                     if (!is_column_full(config, toCol)) {
41                         let newConfig = move_container(config, fromCol,
42                                                         toCol);
43                         const newConfigStr = JSON.stringify(newConfig);
44                         const currentConfigStr = JSON.stringify(config);
45
46                         if (!visited.has(newConfigStr) && newConfigStr
47                             !== currentConfigStr) {
```

```

45         moves.push(newConfig);
46     }
47 }
48 }
49 }
50 }
51 }
52 return moves;
53 }
54
55 function get_top_container(config, col) {
56     for (let i = 0; i < m; i++) {
57         let index = col * m + i;
58         if (config[index] !== 0) {
59             return { value: config[index], index };
60         }
61     }
62     return null;
63 }
64
65 function is_column_full(config, col) {
66     for (let i = 0; i < m; i++) {
67         let index = col * m + i;
68         if (config[index] == 0) {
69             return false;
70         }
71     }
72     return true;
73 }
74
75 function move_container(config, fromCol, toCol) {
76     const top = get_top_container(config, fromCol);
77     if (!top) return config;
78
79     let configArr = config.slice();
80
81     let destIndex = toCol * m + m - 1;
82     while (destIndex >= toCol * m && configArr[destIndex] !== 0) {
83         destIndex--;
84     }
85
86     if (destIndex >= toCol * m) {
87         configArr[top.index] = 0;
88         configArr[destIndex] = top.value;
89     } else {
90         return config;
91     }
92
93     return configArr;
94 }
95
96 function is_sorted(config) {
97     for (let i = 0; i < n; i++) {
98         let temp = 0;
99         for (let j = 0; j < m; j++) {
100             let index = i * m + j;
101             if (config[index] >= temp) {

```

```

102         temp = config[index];
103     } else {
104         return false;
105     }
106 }
107 }
108 return true;
109 }
110
111 function reconstruct_path(config) {
112     let path = [];
113     let currentConfigStr = JSON.stringify(config);
114     while (currentConfigStr !== null) {
115         const currentConfig = JSON.parse(currentConfigStr);
116         path.unshift(currentConfig);
117         const parentConfigStr = parentMap.get(currentConfigStr);
118         if (parentConfigStr) {
119             currentConfigStr = parentConfigStr;
120         } else {
121             currentConfigStr = null;
122         }
123     }
124     return path;
125 }
126
127 function start() {
128     console.log("Initial Configuration:");
129     print_config(initial_config);
130
131     if (is_sorted(initial_config)) {
132         console.log("Initial configuration is already sorted.");
133         return;
134     }
135
136     const startTime = Date.now(); // Start time measurement
137     queue.push(initial_config);
138     visited.add(JSON.stringify(initial_config));
139     parentMap.set(JSON.stringify(initial_config), null);
140     console.log("Initialized BFS with initial configuration.\n");
141
142     while (queue.length > 0) {
143         let currentConfig = queue.shift();
144
145         if (is_sorted(currentConfig)) {
146             const endTime = Date.now(); // End time measurement
147             console.log("Found a sorted configuration:");
148             print_config(currentConfig);
149
150             let path = reconstruct_path(currentConfig);
151             console.log("Number of moves to reach sorted configuration:",
152                 path.length - 1);
153             console.log("Path from initial to sorted configuration:\n");
154             path.forEach((step, index) => {
155                 console.log('Move ${index}:');
156                 print_config(step);
157             });
158             console.log('Time taken: ${(endTime - startTime) / 1000}

```

```

158         seconds');
159         found = true;
160         return;
161     }
162
163     let possibleMoves = generate_moves(currentConfig);
164     for (let move of possibleMoves) {
165         const moveStr = JSON.stringify(move);
166         if (!visited.has(moveStr)) {
167             visited.add(moveStr);
168             parentMap.set(moveStr, JSON.stringify(currentConfig));
169             queue.push(move);
170         }
171     }
172
173     // If a sorted configuration is not found
174     if (!found) {
175         const endTime = Date.now(); // End time measurement
176         console.log("No sorted configuration found.");
177         console.log('Time taken: ${(endTime - startTime) / 1000} seconds');
178     }
179 }
180
181 function init() {
182     console.log("Initial Configuration:");
183     print_config(initial_config);
184     console.log("Is the initial configuration sorted?", is_sorted(
185         initial_config));
186     console.log("Starting the sorting process...\n");
187     start();
188 }
189
190 init();
191
192 module.exports = {
193     init,
194     initial_config,
195     is_sorted,
196     generate_moves,
197     move_container,
198     get_top_container,
199     is_column_full,
200 };

```

B.2 BFS Algorithm in C

This code utilizes a custom library that will not be included in this appendix as it is not directly relevant to the algorithm. The library is a graph library similar to JavaScript but slightly more optimized for the BRP. Any other graph library or graph data structure should be compatible with the code, provided the necessary adaptations are made.

The C library and algorithm were developed by Jiaxu Yang and Yangqi Zheng, with the assistance of AI tools such as OpenAI o1-preview [26][25] for certain code fragments.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <stdbool.h>
5  #include "graphlib.h"
6
7  #define MAX_QUEUE_SIZE 1000000
8  #define HASH_TABLE_SIZE 100003
9
10 typedef struct {
11     int* config;
12     int size;
13 } Configuration;
14
15 typedef struct QueueNode {
16     Configuration config;
17     struct QueueNode* next;
18 } QueueNode;
19
20 typedef struct {
21     QueueNode* front;
22     QueueNode* rear;
23 } Queue;
24
25 typedef struct HashNode {
26     Configuration config;
27     Configuration parent;
28     struct HashNode* next;
29 } HashNode;
30
31 their parents
32 HashNode* hash_table[HASH_TABLE_SIZE];
33
34 void enqueue(Queue* queue, Configuration* config);
35 bool dequeue(Queue* queue, Configuration* config);
36
37 unsigned long hash_configuration(Configuration* config);
38 bool configurations_equal(Configuration* a, Configuration* b);
39
40 bool is_visited(Configuration* config);
41 void mark_visited(Configuration* config, Configuration* parent);
42
43 bool is_sorted(Configuration* config, int m, int n);
44 Configuration* generate_moves(Configuration* config, int m, int n, int*
    move_count);
```

```

45 Configuration move_container(Configuration* config, int fromCol, int
    toCol, int m, int n);
46
47 void print_configuration(Configuration* config, int m, int n);
48 void reconstruct_path(Configuration* end_config, int m, int n,
    Configuration* initial_config);
49
50 Configuration create_configuration(int* array, int size);
51 void free_configuration(Configuration* config);
52
53 int get_top_container(Configuration* config, int col, int m, int n);
54 bool is_column_full(Configuration* config, int col, int m, int n);
55
56 int main() {
57     int m, n;
58     printf("Enter the number of rows(m): ");
59     if (scanf("%d", &m) != 1 || m <= 0) {
60         printf("Invalid input for m.\n");
61         return EXIT_FAILURE;
62     }
63
64     printf("Enter the number of columns(n): ");
65     if (scanf("%d", &n) != 1 || n <= 0) {
66         printf("Invalid input for n.\n");
67         return EXIT_FAILURE;
68     }
69
70     int total = m * n;
71     int* initial_array = malloc(total * sizeof(int));
72     if (initial_array == NULL) {
73         printf("Error allocating memory for the initial configuration.\n");
74         return EXIT_FAILURE;
75     }
76
77     printf("Enter the elements of the initial configuration (separated by
        spaces):\n");
78     for (int i = 0; i < total; i++) {
79         if (scanf("%d", &initial_array[i]) != 1) {
80             printf("Invalid input at index %d.\n", i);
81             free(initial_array);
82             return EXIT_FAILURE;
83         }
84     }
85
86     int c;
87     while ((c = getchar()) != '\n' && c != EOF);
88
89     Configuration initial_config = create_configuration(initial_array,
        total);
90     free(initial_array);
91
92     printf("Initial configuration:\n");
93     print_configuration(&initial_config, m, n);
94
95     if (is_sorted(&initial_config, m, n)) {
96         printf("The initial configuration is already sorted.\n");

```

```

97     printf("Press Enter to exit...\n");
98     getchar();
99     free_configuration(&initial_config);
100     return EXIT_SUCCESS;
101 }
102
103
104 Queue queue = { NULL, NULL };
105 enqueue(&queue, &initial_config);
106
107 mark_visited(&initial_config, NULL);
108
109 bool found = false;
110 Configuration end_config = { NULL, 0 };
111
112 while (queue.front != NULL) {
113     Configuration current_config;
114     if (!dequeue(&queue, &current_config)) {
115         break;
116     }
117
118     if (is_sorted(&current_config, m, n)) {
119         printf("Sorted configuration found!\n");
120         end_config = current_config;
121         found = true;
122         break;
123     }
124
125     int move_count = 0;
126     Configuration* moves = generate_moves(&current_config, m, n, &
        move_count);
127
128     for (int i = 0; i < move_count; i++) {
129         if (!is_visited(&moves[i])) {
130             mark_visited(&moves[i], &current_config);
131             enqueue(&queue, &moves[i]);
132         } else {
133             free_configuration(&moves[i]);
134         }
135     }
136
137     free(moves);
138     // Do not free current_config here, as it may be needed to
139     // reconstruct the path
140     // free_configuration(&current_config);
141 }
142
143 if (found) {
144     reconstruct_path(&end_config, m, n, &initial_config);
145     free_configuration(&end_config);
146 } else {
147     printf("No sorted configuration found.\n");
148 }
149
150 for (int i = 0; i < HASH_TABLE_SIZE; i++) {
151     HashNode* current = hash_table[i];
152     while (current != NULL) {

```

```

152         HashNode* temp = current;
153         current = current->next;
154         free_configuration(&temp->config);
155         free_configuration(&temp->parent);
156         free(temp);
157     }
158 }
159
160 free_configuration(&initial_config);
161
162 printf("Press Enter to exit...\n");
163 getchar();
164
165 return EXIT_SUCCESS;
166 }
167
168 //////////////////////////////////////
169 //      Memory Functions      //
170 //////////////////////////////////////
171
172 Configuration create_configuration(int* array, int size) {
173     Configuration config;
174     config.size = size;
175     config.config = malloc(size * sizeof(int));
176     if (config.config == NULL) {
177         fprintf(stderr, "Error allocating memory for the configuration.\n");
178         exit(EXIT_FAILURE);
179     }
180     memcpy(config.config, array, size * sizeof(int));
181     return config;
182 }
183
184 void free_configuration(Configuration* config) {
185     if (config->config != NULL) {
186         free(config->config);
187         config->config = NULL;
188         config->size = 0;
189     }
190 }
191
192 //////////////////////////////////////
193 //      Queue Functions      //
194 //////////////////////////////////////
195
196 void enqueue(Queue* queue, Configuration* config) {
197     QueueNode* new_node = malloc(sizeof(QueueNode));
198     if (new_node == NULL) {
199         fprintf(stderr, "Error allocating memory for the queue node.\n");
200         exit(EXIT_FAILURE);
201     }
202     new_node->config = create_configuration(config->config, config->size);
203     new_node->next = NULL;
204
205     if (queue->rear == NULL) {
206         queue->front = queue->rear = new_node;

```

```

207         return;
208     }
209
210     queue->rear->next = new_node;
211     queue->rear = new_node;
212 }
213
214 bool dequeue(Queue* queue, Configuration* config) {
215     if (queue->front == NULL) {
216         return false;
217     }
218
219     QueueNode* temp = queue->front;
220     *config = temp->config;
221
222     queue->front = queue->front->next;
223     if (queue->front == NULL) {
224         queue->rear = NULL;
225     }
226
227     free(temp);
228
229     return true;
230 }
231
232 ////////////////////////////////////////////////////
233 //          Hash Functions          //
234 ////////////////////////////////////////////////////
235
236 unsigned long hash_configuration(Configuration* config) {
237     unsigned long hash = 5381;
238     for (int i = 0; i < config->size; i++) {
239         hash = ((hash << 5) + hash) + config->config[i];
240     }
241     return hash % HASH_TABLE_SIZE;
242 }
243
244 bool configurations_equal(Configuration* a, Configuration* b) {
245     if (a->size != b->size) {
246         return false;
247     }
248     for (int i = 0; i < a->size; i++) {
249         if (a->config[i] != b->config[i]) {
250             return false;
251         }
252     }
253     return true;
254 }
255
256 ////////////////////////////////////////////////////
257 // Visited Set and Parent Map Functions //
258 ////////////////////////////////////////////////////
259
260 bool is_visited(Configuration* config) {
261     unsigned long hash_index = hash_configuration(config);
262     HashNode* current = hash_table[hash_index];
263     while (current != NULL) {

```

```

264         if (configurations_equal(&current->config, config)) {
265             return true;
266         }
267         current = current->next;
268     }
269     return false;
270 }
271
272 void mark_visited(Configuration* config, Configuration* parent) {
273     unsigned long hash_index = hash_configuration(config);
274     HashNode* new_node = malloc(sizeof(HashNode));
275     if (new_node == NULL) {
276         fprintf(stderr, "Error allocating memory for HashNode.\n");
277         exit(EXIT_FAILURE);
278     }
279     new_node->config = create_configuration(config->config, config->size)
        ;
280
281     if (parent != NULL) {
282         new_node->parent = create_configuration(parent->config, parent->
            size);
283     } else {
284         new_node->parent.config = NULL;
285         new_node->parent.size = 0;
286     }
287     new_node->next = hash_table[hash_index];
288     hash_table[hash_index] = new_node;
289 }
290
291 ////////////////////////////////////////////
292 //           BFS Functions           //
293 ////////////////////////////////////////////
294
295 bool is_sorted(Configuration* config, int m, int n) {
296     for (int col = 0; col < n; col++) {
297         int prev = 0;
298         for (int row = 0; row < m; row++) {
299             int index = row * n + col;
300             int value = config->config[index];
301             if (value < prev) {
302                 return false;
303             }
304             prev = value;
305         }
306     }
307     return true;
308 }
309
310 Configuration* generate_moves(Configuration* config, int m, int n, int*
    move_count) {
311     int max_moves = n * (n - 1);
312     Configuration* moves = malloc(max_moves * sizeof(Configuration));
313     if (moves == NULL) {
314         fprintf(stderr, "Error allocating memory for moves.\n");
315         exit(EXIT_FAILURE);
316     }
317     *move_count = 0;

```

```

318
319     for (int fromCol = 0; fromCol < n; fromCol++) {
320         int fromIndex = get_top_container(config, fromCol, m, n);
321         if (fromIndex != -1) {
322             for (int toCol = 0; toCol < n; toCol++) {
323                 if (toCol != fromCol && !is_column_full(config, toCol, m,
324                     n)) {
325                     Configuration new_config = move_container(config,
326                         fromCol, toCol, m, n);
327                     if (!configurations_equal(config, &new_config)) {
328                         moves[*move_count] = new_config;
329                         (*move_count)++;
330                     } else {
331                         free_configuration(&new_config);
332                     }
333                 }
334             }
335         }
336     }
337     return moves;
338 }
339
340 Configuration move_container(Configuration* config, int fromCol, int
341     toCol, int m, int n) {
342     Configuration new_config = create_configuration(config->config,
343         config->size);
344
345     int fromIndex = get_top_container(&new_config, fromCol, m, n);
346     if (fromIndex == -1) {
347         return new_config;
348     }
349
350     int toIndex = -1;
351     for (int row = m - 1; row >= 0; row--) {
352         int index = row * n + toCol;
353         if (new_config.config[index] == 0) {
354             toIndex = index;
355             break;
356         }
357     }
358
359     if (toIndex != -1) {
360         new_config.config[toIndex] = new_config.config[fromIndex];
361         new_config.config[fromIndex] = 0;
362     }
363
364     return new_config;
365 }
366
367 int get_top_container(Configuration* config, int col, int m, int n) {
368     for (int row = 0; row < m; row++) {
369         int index = row * n + col;
370         if (config->config[index] != 0) {
371             return index;
372         }
373     }
374     return -1;

```

```

371 }
372
373 bool is_column_full(Configuration* config, int col, int m, int n) {
374     for (int row = 0; row < m; row++) {
375         int index = row * n + col;
376         if (config->config[index] == 0) {
377             return false;
378         }
379     }
380     return true;
381 }
382
383 ////////////////////////////////////////////////////
384 //           Helper Functions           //
385 ////////////////////////////////////////////////////
386
387 void print_configuration(Configuration* config, int m, int n) {
388     for (int row = 0; row < m; row++) {
389         for (int col = 0; col < n; col++) {
390             int index = row * n + col;
391             printf("%d", config->config[index]);
392         }
393         printf("\n");
394     }
395     printf("\n");
396 }
397
398 void reconstruct_path(Configuration* end_config, int m, int n,
399 Configuration* initial_config) {
400     printf("Reconstructing the path to the sorted configuration:\n");
401
402     int max_path_length = MAX_QUEUE_SIZE;
403     Configuration* path = malloc(max_path_length * sizeof(Configuration))
404     ;
405     if (path == NULL) {
406         fprintf(stderr, "Error allocating memory for the path.\n");
407         exit(EXIT_FAILURE);
408     }
409     int path_length = 0;
410
411     Configuration current = *end_config;
412     while (true) {
413         if (path_length >= max_path_length) {
414             fprintf(stderr, "Error: Path length exceeds maximum allowed
415 size.\n");
416             exit(EXIT_FAILURE);
417         }
418
419         path[path_length] = create_configuration(current.config, current.
420 size);
421         path_length++;
422
423         if (configurations_equal(&current, initial_config)) {
424             break;
425         }
426
427         unsigned long hash_index = hash_configuration(&current);

```

```

424     HashNode* node = hash_table[hash_index];
425     bool found_parent = false;
426     while (node != NULL) {
427         if (configurations_equal(&node->config, &current)) {
428             if (node->parent.config == NULL || node->parent.size ==
429                 0) {
430                 found_parent = false;
431                 break;
432             }
433             current = node->parent;
434             found_parent = true;
435             break;
436         }
437         node = node->next;
438     }
439     if (!found_parent) {
440         fprintf(stderr, "Error: Parent configuration not found.\n");
441         break;
442     }
443 }
444
445 printf("Number of moves to reach the sorted configuration: %d\n",
446        path_length - 1);
447 for (int i = path_length - 1; i >= 0; i--) {
448     printf("Move %d:\n", path_length - i - 1);
449     print_configuration(&path[i], m, n);
450     free_configuration(&path[i]);
451 }
452 free(path);
453 }

```

B.3 Additional data for code performance

3 rows and 3 columns configuration instances				
Initial Configuration	No. of Empty Spaces	Sorted Configuration	No. of Moves	Time (s)
0, 4, 6, 2, 1, 3, 0, 0, 5	3	2, 4, 6, 0, 1, 3, 0, 0, 5	1	0.001
1, 2, 4, 0, 5, 3, 0, 0, 6	3	1, 2, 4, 0, 0, 3, 0, 5, 6	1	0.001
0, 0, 2, 5, 3, 4, 0, 1, 6	3	0, 1, 2, 0, 3, 4, 0, 5, 6	2	0.001
0, 2, 3, 6, 1, 4, 0, 0, 5	3	0, 2, 3, 0, 1, 4, 0, 5, 6	3	0.001
0, 2, 1, 6, 3, 5, 0, 0, 4	3	0, 0, 6, 0, 3, 5, 1, 2, 4	3	0.001
3, 5, 4, 2, 1, 6, 0, 0, 0	3	0, 0, 4, 0, 5, 6, 1, 2, 3	4	0.001
3, 4, 2, 5, 1, 6, 0, 0, 0	3	0, 0, 2, 0, 1, 6, 3, 4, 5	4	0.001
0, 3, 1, 0, 0, 5, 2, 6, 4	3	0, 0, 6, 0, 2, 5, 1, 3, 4	5	0.001
6, 2, 1, 0, 5, 4, 0, 0, 3	3	0, 5, 6, 0, 0, 4, 1, 2, 3	5	0.001
0, 6, 4, 0, 2, 1, 0, 5, 3	3	1, 2, 4, 0, 5, 6, 0, 0, 3	5	0.001
7, 2, 3, 6, 1, 4, 0, 0, 5	2	No sorted configuration	0	0.001
7, 3, 1, 0, 0, 5, 2, 6, 4	2	No sorted configuration	0	0.001
7, 4, 6, 2, 1, 3, 0, 0, 5	2	No sorted configuration	0	0.001
1, 2, 4, 7, 5, 3, 0, 0, 6	2	No sorted configuration	0	0.001
0, 7, 2, 5, 3, 4, 0, 1, 6	2	No sorted configuration	0	0.001
6, 2, 1, 7, 5, 4, 0, 0, 3	2	No sorted configuration	0	0.001
7, 2, 1, 6, 3, 5, 0, 0, 4	2	No sorted configuration	0	0.001
7, 6, 4, 0, 2, 1, 0, 5, 3	2	No sorted configuration	0	0.001
3, 5, 4, 2, 1, 6, 0, 0, 7	2	1, 2, 4, 0, 5, 6, 0, 3, 7	5	0.001
3, 4, 2, 5, 1, 6, 0, 0, 7	2	0, 0, 2, 4, 5, 6, 1, 3, 7	5	0.001

Table 6: Additional data for the configuration with 3 columns and 3 rows the structure and division of the data is the same as Table 1, in the main paper, except we have 10 configurations each division.

3 rows and 4 columns configuration instances				
Initial Configuration	No. of Empty Spaces	Sorted Configuration	No. of Moves	Time (s)
4, 5, 7, 6, 1, 8, 0, 3, 2, 0, 0, 0	4	4, 5, 7, 0, 1, 8, 0, 0, 2, 0, 3, 6	2	0.001
1, 7, 3, 0, 6, 2, 0, 4, 5, 0, 0, 8	4	0, 0, 3, 0, 0, 2, 1, 4, 5, 6, 7, 8	3	0.001
4, 1, 8, 6, 5, 7, 0, 0, 0, 0, 3, 2	4	0, 1, 8, 4, 5, 7, 0, 3, 6, 0, 0, 2	3	0.001
3, 4, 2, 8, 1, 5, 0, 0, 7, 0, 0, 6	4	0, 0, 8, 0, 1, 5, 2, 3, 7, 0, 4, 6	4	0.002
0, 4, 2, 8, 7, 5, 0, 3, 6, 0, 0, 1	4	0, 0, 2, 0, 4, 5, 1, 3, 6, 0, 7, 8	4	0.006
0, 0, 0, 3, 2, 6, 0, 1, 5, 4, 8, 7	4	0, 3, 8, 0, 2, 6, 0, 1, 5, 0, 4, 7	4	0.005
4, 7, 2, 0, 1, 3, 0, 6, 5, 0, 0, 8	4	0, 0, 2, 0, 1, 3, 0, 4, 5, 6, 7, 8	4	0.002
8, 6, 1, 0, 0, 0, 3, 7, 2, 0, 4, 5	4	0, 0, 1, 6, 7, 8, 0, 0, 2, 3, 4, 5	4	0.002
0, 0, 4, 6, 5, 2, 3, 8, 7, 0, 0, 1	4	1, 3, 4, 0, 0, 2, 5, 6, 7, 0, 0, 8	5	0.012
0, 0, 6, 4, 8, 2, 0, 3, 1, 0, 7, 5	4	2, 4, 6, 3, 7, 8, 0, 0, 1, 0, 0, 5	6	0.020
4, 5, 7, 6, 1, 8, 9, 3, 2, 0, 0, 0	3	4, 5, 7, 0, 1, 8, 0, 0, 2, 3, 6, 9	3	0.001
4, 1, 8, 6, 5, 7, 0, 0, 9, 0, 3, 2	3	0, 1, 8, 4, 5, 7, 3, 6, 9, 0, 0, 2	3	0.001
0, 0, 9, 3, 2, 6, 0, 1, 5, 4, 8, 7	3	3, 8, 9, 0, 2, 6, 0, 1, 5, 0, 4, 7	4	0.003
8, 6, 1, 0, 0, 9, 3, 7, 2, 0, 4, 5	3	0, 0, 1, 2, 7, 9, 0, 6, 8, 3, 4, 5	5	0.007
1, 7, 3, 9, 6, 2, 0, 4, 5, 0, 0, 8	3	0, 6, 9, 0, 0, 2, 1, 4, 5, 3, 7, 8	5	0.004
9, 4, 2, 8, 7, 5, 0, 3, 6, 0, 0, 1	3	0, 0, 2, 0, 4, 5, 1, 3, 6, 7, 8, 9	5	0.008
4, 7, 2, 9, 1, 3, 0, 6, 5, 0, 0, 8	3	6, 7, 9, 0, 1, 3, 0, 0, 5, 2, 4, 8	6	0.010
3, 4, 2, 8, 1, 5, 0, 9, 7, 0, 0, 6	3	0, 8, 9, 0, 1, 5, 2, 4, 7, 0, 3, 6	6	0.012
0, 9, 6, 4, 8, 2, 0, 3, 1, 0, 7, 5	3	2, 4, 6, 7, 8, 9, 0, 0, 1, 0, 3, 5	7	0.026
0, 9, 4, 6, 5, 2, 3, 8, 7, 0, 0, 1	3	1, 3, 4, 0, 0, 2, 5, 6, 7, 0, 8, 9	8	0.065

Table 7: Additional data for the configuration with 4 columns and 3 rows the structure and division of the data is the same as Table 2, in the main paper, except we have 10 configurations each division.

4 rows and 3 columns configuration instances				
Initial Configuration	No. of Empty Spaces	Sorted Configuration	No. of Moves	Time (s)
0, 3, 2, 8, 4, 1, 7, 6, 0, 0, 0, 5	4	0, 0, 7, 8, 0, 2, 3, 6, 0, 1, 4, 5	6	0.003
8, 4, 1, 6, 0, 7, 5, 3, 0, 0, 0, 2	4	0, 0, 1, 6, 0, 0, 2, 3, 4, 5, 7, 8	7	0.003
7, 1, 5, 3, 0, 4, 6, 8, 0, 0, 0, 2	4	0, 0, 5, 7, 3, 4, 6, 8, 0, 0, 1, 2	7	0.003
1, 5, 8, 2, 0, 7, 4, 6, 0, 0, 0, 3	4	0, 5, 7, 8, 0, 2, 4, 6, 0, 0, 1, 3	8	0.008
0, 7, 1, 4, 0, 8, 2, 6, 0, 0, 5, 3	4	4, 5, 7, 8, 0, 1, 2, 6, 0, 0, 0, 3	8	0.011
6, 4, 2, 1, 0, 8, 3, 7, 0, 0, 0, 5	4	0, 0, 6, 8, 1, 2, 3, 7, 0, 0, 4, 5	8	0.007
6, 7, 1, 5, 0, 3, 8, 2, 0, 0, 0, 4	4	0, 0, 1, 5, 0, 0, 7, 8, 2, 3, 4, 6	8	0.009
0, 5, 6, 3, 1, 7, 8, 2, 0, 0, 0, 4	4	0, 0, 2, 3, 5, 6, 7, 8, 0, 0, 1, 4	10	0.023
1, 8, 3, 6, 2, 5, 7, 4, 0, 0, 0, 0	4	0, 1, 3, 6, 4, 5, 7, 8, 0, 0, 0, 2	11	0.038
5, 8, 7, 2, 0, 6, 3, 1, 0, 0, 0, 4	4	3, 6, 7, 8, 0, 0, 0, 1, 0, 2, 4, 5	12	0.134
1, 5, 8, 2, 9, 7, 4, 6, 0, 0, 0, 3	3	No sorted configuration	0	0.020
9, 7, 1, 4, 0, 8, 2, 6, 0, 0, 5, 3	3	No sorted configuration	0	0.020
6, 4, 2, 1, 9, 8, 3, 7, 0, 0, 0, 5	3	No sorted configuration	0	0.027
5, 8, 7, 2, 9, 6, 3, 1, 0, 0, 0, 4	3	No sorted configuration	0	0.022
9, 5, 6, 3, 1, 7, 8, 2, 0, 0, 0, 4	3	No sorted configuration	0	0.022
6, 7, 1, 5, 9, 3, 8, 2, 0, 0, 0, 4	3	No sorted configuration	0	0.027
8, 4, 1, 6, 9, 7, 5, 3, 0, 0, 0, 2	3	No sorted configuration	0	0.019
9, 3, 2, 8, 4, 1, 7, 6, 0, 0, 0, 5	3	No sorted configuration	0	0.021
7, 1, 5, 3, 9, 4, 6, 8, 0, 0, 0, 2	3	No sorted configuration	0	0.021
1, 8, 3, 6, 2, 5, 7, 4, 0, 0, 0, 9	3	1, 2, 3, 6, 0, 0, 0, 4, 5, 7, 8, 9	12	0.009

Table 8: Additional data for the configuration with 3 columns and 4 rows the structure and division of the data is the same as Table 3, in the main paper, except we have 10 configurations each division.

4 rows and 4 columns configuration instances				
Initial Configuration	No. of Empty Spaces	Sorted Configuration	No. of Moves	Time (s)
5, 6, 2, 1, 8, 4, 9, 10, 0, 11, 3, 7, 0, 0, 0, 12	4	0, 0, 0, 1, 3, 4, 9, 10, 0, 2, 6, 7, 5, 8, 11, 12	6	0.043
0, 3, 5, 2, 4, 8, 1, 7, 0, 11, 6, 12, 0, 0, 9, 10	4	0, 0, 8, 11, 0, 0, 1, 7, 2, 5, 6, 12, 3, 4, 9, 10	7	0.387
0, 1, 7, 12, 3, 4, 10, 2, 0, 0, 5, 11, 0, 8, 6, 9	4	0, 1, 7, 12, 0, 3, 8, 10, 2, 4, 5, 11, 0, 0, 6, 9	7	0.250
3, 1, 5, 4, 0, 12, 8, 10, 0, 0, 7, 9, 0, 11, 6, 2	4	0, 6, 11, 12, 4, 5, 8, 10, 1, 3, 7, 9, 0, 0, 0, 2	8	1.099
3, 10, 8, 6, 0, 4, 9, 1, 0, 0, 11, 5, 0, 2, 7, 12	4	0, 0, 0, 6, 8, 9, 10, 11, 0, 3, 4, 5, 1, 2, 7, 12	10	69.334
9, 6, 3, 12, 0, 11, 10, 7, 4, 2, 8, 5, 0, 0, 0, 1	4	8, 10, 11, 12, 0, 2, 4, 7, 0, 0, 0, 5, 1, 3, 6, 9	10	152.804
4, 1, 12, 6, 9, 5, 3, 2, 0, 0, 0, 11, 0, 8, 10, 7	4	3, 5, 10, 12, 0, 0, 0, 2, 6, 8, 9, 11, 0, 1, 4, 7	11	566.058
10, 1, 8, 7, 0, 2, 11, 9, 0, 12, 6, 4, 0, 0, 5, 3	4	0, 0, 0, 7, 3, 5, 6, 9, 0, 1, 2, 4, 8, 10, 11, 12	13	Greater than 3600
1, 10, 4, 7, 3, 11, 9, 2, 0, 8, 12, 5, 0, 0, 0, 6	4	2, 3, 4, 7, 5, 8, 9, 10, 0, 0, 11, 12, 0, 0, 1, 6	13	Greater than 3600
0, 11, 1, 3, 0, 2, 6, 5, 9, 10, 8, 4, 0, 0, 12, 7	4	0, 0, 1, 3, 8, 10, 11, 12, 0, 0, 2, 4, 5, 6, 7, 9	13	Greater than 3600
5, 6, 2, 1, 8, 4, 9, 10, 13, 11, 3, 7, 0, 0, 0, 12	3	No sorted configuration	0	Greater than 3600
3, 1, 5, 4, 13, 12, 8, 10, 0, 0, 7, 9, 0, 11, 6, 2	3	No sorted configuration	0	Greater than 3600
13, 3, 5, 2, 4, 8, 1, 7, 0, 11, 6, 12, 0, 0, 9, 10	3	No sorted configuration	0	Greater than 3600
10, 1, 8, 7, 0, 2, 11, 9, 13, 12, 6, 4, 0, 0, 5, 3	3	No sorted configuration	0	Greater than 3600
1, 10, 4, 7, 3, 11, 9, 2, 13, 8, 12, 5, 0, 0, 0, 6	3	No sorted configuration	0	Greater than 3600
9, 6, 3, 12, 13, 11, 10, 7, 4, 2, 8, 5, 0, 0, 0, 1	3	No sorted configuration	0	Greater than 3600
4, 1, 12, 6, 9, 5, 3, 2, 0, 0, 13, 11, 0, 8, 10, 7	3	No sorted configuration	0	Greater than 3600
13, 11, 1, 3, 0, 2, 6, 5, 9, 10, 8, 4, 0, 0, 12, 7	3	No sorted configuration	0	Greater than 3600
3, 10, 8, 6, 13, 4, 9, 1, 0, 0, 11, 5, 0, 2, 7, 12	3	No sorted configuration	0	Greater than 3600
13, 1, 7, 12, 3, 4, 10, 2, 0, 0, 5, 11, 0, 8, 6, 9	3	No sorted configuration	0	Greater than 3600

Table 9: Additional data for the configuration with 3 columns and 3 rows the structure and division of the data is the same as Table 4, in the main paper, except we have 10 configurations each division.